Windows CMD Mastery

From Zero to Expert Windows Administration, Automation, Internals & Defensive Security Research

A Hands-On Command Prompt, Batch Scripting, Windows Administration and Security Research Field Guide

Volume 2

Parts VI-XVIII: Advanced Batch, Filesystem, Processes, Services, Tokens, Registry, Tasks, Networking, Firewall, Ever
Labs 26-40, 66-70, 81-112, 151-165

Copyright and Disclaimer

Copyright (c) Windows CMD Mastery. All rights reserved. This manuscript is educational material. Every command is provided for use on systems you own or are explicitly authorized to administer. Run every lab inside an isolated virtual machine with a snapshot taken first. Commands that change system configuration, security settings, disks, boot data or encryption state are marked DANGER and can cause data loss. The author and publisher accept no liability for damage resulting from use of this material. Windows behavior differs between versions, editions and builds; verify each command against current Microsoft Learn documentation for your exact build. No malware, credential theft, evasion or unauthorized-access content is included; security chapters use benign simulations only.

How to Use This Book

Read in order the first time. Type every example; do not paste. Every lab lists its own objective, prerequisites, expected result and cleanup. Work inside a disposable VM.

Legend	Meaning
[B] Beginner	No prior knowledge assumed
[I] Intermediate	Needs the earlier chapters
[A] Advanced	Needs parser and batch fundamentals
[X] Expert	Internals, edge cases, design trade-offs

Callout	Meaning
SAFETY (green)	How to keep a lab contained and reversible
DANGER / CAUTION (red)	Can destroy data, change boot or security state, or needs a snapshot
SECURITY (orange)	Defensive implication, abuse potential, detection note
NOTE (blue)	Clarification or version note

Table of Contents

Copyright and Disclaimer 2

How to Use This Book 2

Table of Contents 3

Part VI - Advanced Batch Engineering 7

Chapter 21. A Professional Script Skeleton 9

21.1 Design goals . . . 9
21.2 Exit-code convention used in this book . . . 9
21.3 The skeleton . . . 9
21.4 Walkthrough . . . 11
21.5 Limits of batch functions . . . 12

Chapter 22. Engineering Patterns 13

22.1 Configuration files . . . 13
22.2 Validation helpers . . . 13
22.3 Backup, change, verify, rollback . . . 13
22.4 Temporary directories . . . 14
22.5 Stale-lock detection . . . 14
22.6 Structured output . . . 15
22.7 Timestamps . . . 15
22.8 Colored output and progress . . . 15
22.9 Administrator detection (and why it is only a convenience) . . . 15
22.10 Self-diagnostics . . . 16

Part VII - The Windows Filesystem 19

Chapter 23. NTFS Concepts 21

23.1 Drives, volumes, paths . . . 21
23.2 NTFS building blocks . . . 21
23.3 Attributes . . . 21
23.4 Links and reparse points . . . 22
23.5 Alternate data streams (concept) . . . 22

23.6 Compression, encryption, quotas . . . 22
23.7 SIDs, ACLs and how access is decided . . . 23

Chapter 24. Permissions and Filesystem Command Cards 24

Part VIII - Windows Processes 35

Chapter 25. Process Concepts and Inspection 37

25.1 What a process is . . . 37
25.2 Tools available from CMD . . . 37

Chapter 26. Investigating a Process 41

26.1 The five questions . . . 41
26.2 Evidence commands . . . 41
26.3 Expected parent/child relationships (Windows 10/11) . . . 41
26.4 What a benign chain looks like . . . 41

Part IX - Windows Services 45

Chapter 27. Services Explained 47

27.1 What a service is . . . 47
27.2 Why services matter to security . . . 47

Chapter 28. Service Labs 50

Part X - Users, Groups and Security Tokens 53

Chapter 29. Identity, Tokens and UAC 55

29.1 Accounts and groups . . . 55
29.2 The access token . . . 55
29.3 UAC and the two tokens . . . 55
29.4 Reading whoami /all line by line . . . 56

Part XI - The Windows Registry 61

Chapter 30. Registry Architecture 63

30.1 What the registry is . . . 63
30.2 The five root keys and the real hives . . . 63
30.3 Value types . . . 63

30.4 32-bit and 64-bit views, and virtualization . . . 64
30.5 Permissions and risk . . . 64
30.6 Backup and restore options . . . 64
30.7 Security-relevant locations (audit checklist) . . . 64

Part XII - Scheduled Tasks 69

Chapter 31. Task Scheduler Architecture and schtasks 71

31.1 Architecture . . . 71
31.2 Investigating a suspicious task . . . 72

Part XIII - Windows Networking 74

Chapter 32. Networking Foundations 76

32.1 The layers you actually troubleshoot . . . 76
32.2 Core vocabulary . . . 76
32.3 TCP versus UDP, and socket states . . . 76
32.4 How a packet leaves your machine . . . 77
32.5 Modern alternatives and deprecated tools . . . 77

Chapter 33. Networking Command Cards 78

Chapter 34. Troubleshooting Decision Trees 83

34.1 The bottom-up ladder . . . 83
34.2 "The browser works but DNS commands fail" (and its mirror) . . . 83
34.3 Other common scenarios . . . 83

Part XIV - Windows Firewall 89

Chapter 35. Windows Defender Firewall from CMD 91

35.1 Concepts . . . 91
35.2 Inspecting safely (read-only) . . . 91
35.3 Controlled changes (modify protection) . . . 91
35.4 Auditing a firewall configuration (defender checklist) . . . 92

Part XV - Windows Event Logging 94

Chapter 36. Event Log Architecture and wevtutil 96

36.1 Architecture . . . 96
36.2 Event XML structure . . . 96
36.3 Important events (reference) . . . 97

Part XVI - Windows Defender 102

Chapter 37. Defender Administration and Investigation 104

37.1 Components and commands . . . 104
37.2 What to check in a health review . . . 104
37.3 Safe testing . . . 105

Part XVII - Windows Repair and Recovery 107

Chapter 38. Repair Tools and Boot Architecture 109

38.1 Concepts . . . 109

Part XVIII - Windows Performance 115

Chapter 39. Performance Measurement and Triage 117

39.1 The four resources and what "slow" means . . . 117

Back Matter (Volume 2) 122

Chapter 40. Glossary Additions 124

Chapter 41. Command Index (Volume 2) 125

Chapter 42. References and Verification Notes 127

Chapter 43. What Comes Next 128

Part VI - Advanced Batch Engineering

Writing batch scripts that behave like professional tools: argument parsing, one exit path, logging, rollback, prerequisite checks, structured output.

Chapter 21. A Professional Script Skeleton

21.1 Design goals

A script other people run should be **predictable** (same inputs, same result), **safe by default** (dry run unless told otherwise), **observable** (logs and exit codes), **reversible** (backs up before changing), and **honest** (fails loudly with a clear message rather than limping on).

Batch has no exceptions, no `finally` and no real functions, so each goal is built from conventions:

Goal	Batch convention
Predictable	Validate every argument; never rely on the caller's current directory
Safe by default	`/go` switch required to change anything
Observable	`:log` function, meaningful `exit /b N` codes
Reversible	Backup before change; restore on failure
Single cleanup point	Every path ends at one `:end` label that calls `:cleanup`

21.2 Exit-code convention used in this book

Code	Meaning
0	Success
1	General failure
2	Bad usage or arguments
3	Missing prerequisite (tool, file, folder)
4	Not elevated when elevation is required
5	Unsupported Windows version
9	Another instance is already running

21.3 The skeleton

Save as `skeleton.cmd`. Read the comments first; the walkthrough follows.

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem ============================================================
rem  skeleton.cmd - template for professional batch tools
rem  usage: skeleton.cmd [/go] [/verbose] [/out folder] [/?]
rem ============================================================
set "APP=skeleton"
set "VERSION=1.0.0"
set "SCRIPTDIR=%~dp0"
set "RC=0"
set "GO=0"
set "VERBOSE=0"
set "OUTDIR="
set "LOGFILE="

rem ---------------- argument parsing ----------------
:parse
if "%~1"=="" goto :parsed
if /i "%~1"=="/?"       goto :usage
if /i "%~1"=="/go"      (set "GO=1" & shift & goto :parse)
if /i "%~1"=="/verbose" (set "VERBOSE=1" & shift & goto :parse)
if /i "%~1"=="/out" (
    if "%~2"=="" (echo ERROR: /out needs a folder & set "RC=2" & goto :end)
    set "OUTDIR=%~2"
    shift & shift & goto :parse
)
echo ERROR: unknown option "%~1"
set "RC=2" & goto :usage_rc

:parsed
if not defined OUTDIR set "OUTDIR=%SCRIPTDIR%out"

rem ---------------- startup checks ----------------
call :check_version || (set "RC=5" & goto :end)
call :require_tool findstr || (set "RC=3" & goto :end)
call :lock_acquire || (set "RC=9" & goto :end)
if not exist "%OUTDIR%\" md "%OUTDIR%" 2>nul
if not exist "%OUTDIR%\" (echo ERROR: cannot create "%OUTDIR%" & set "RC=1" & goto :end)
set "LOGFILE=%OUTDIR%\%APP%.log"

call :log INFO "%APP% %VERSION% starting (go=%GO%)"
call :main
set "RC=%ERRORLEVEL%"
goto :end

rem ---------------- main work ----------------
:main
if "%GO%"=="0" (
    call :log INFO "dry run: no changes made (use /go to apply)"
    exit /b 0
)
call :log INFO "doing the real work here"
exit /b 0

rem ---------------- helpers ----------------
:log
rem usage: call :log LEVEL "message"
if "%~1"=="DEBUG" if not "%VERBOSE%"=="1" exit /b 0
```

```
set "T=%TIME: =0%"
if defined LOGFILE (>>"%LOGFILE%" echo %DATE% %T:~0,8% [%~1] %~2)
echo [%~1] %~2
exit /b 0

:require_tool
where /q %~1 && exit /b 0
echo ERROR: required tool not found: %~1
exit /b 1

:check_version
for /f "tokens=3" %%b in ('reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion" /v CurrentBuildNumber 2
if not defined BUILD (echo ERROR: cannot read Windows build & exit /b 1)
if %BUILD% lss 17763 (echo ERROR: Windows build %BUILD% is too old & exit /b 1)
exit /b 0

:lock_acquire
set "LOCKDIR=%TEMP%\%APP%.lock"
mkdir "%LOCKDIR%" 2>nul && (set "HAVELOCK=1" & exit /b 0)
echo ERROR: another instance appears to be running (%LOCKDIR%)
exit /b 1

:cleanup
if defined HAVELOCK rmdir "%LOCKDIR%" 2>nul
exit /b 0

:usage
set "RC=0"
:usage_rc
echo Usage: %~nx0 [/go] [/verbose] [/out folder] [/?]
echo   (default is a dry run; nothing is changed without /go)

:end
call :cleanup
exit /b %RC%
```

21.4 Walkthrough

- `setlocal EnableExtensions EnableDelayedExpansion` keeps changes private and enables `!VAR!` where needed. Beware: with delayed expansion on, `!` characters in arguments and file names are consumed (Chapter 10); this skeleton uses `%~1` for arguments to limit exposure.
- **Argument parser.** A `:parse` loop examines `%~1`, handles switches, and uses `shift` to advance. A switch with a value reads `%~2` and shifts twice. Anything unrecognized is an error: **reject, do not ignore**, because a typo like `/gp` silently running a dry run is bad, and a typo that silently runs the real thing is worse.
- **Single exit path.** Every failure sets `RC` and jumps to `:end`, which calls `:cleanup` exactly once. That is the batch substitute for `finally`.
- `:usage` **falls into** `:usage_rc`: `:usage` sets RC to 0 (the user asked for help), unknown options jump straight to `:usage_rc` with RC already 2.
- `:log` writes to console and file; the `DEBUG` line returns early unless `/verbose` was given. `%TIME: =0%` repairs the blank before single-digit hours.
- `:check_version` reads the build from the registry (locale-independent, unlike parsing `ver`). Build 17763 is Windows 10 1809 / Server 2019; choose your own floor. The caret before `2>nul` is needed because the command sits inside `for /f ... in ('...')`.
- `:lock_acquire` uses `mkdir` atomicity. Its weakness: a crash leaves the lock folder behind. See Section 22.5 for a stale-lock improvement.

> **NOTE.** `if "%GO%"=="0" ( ... )` blocks use `%GO%` safely because GO is never modified inside that block. Inside any block where a variable changes, use `!VAR!`.

21.5 Limits of batch functions

- Subroutines share **one global variable namespace**. Prefix helper variables (`_T`, `_LOG_`) or wrap them in `setlocal`/`endlocal`.
- Returning values means writing a caller-named variable; returning a value across `endlocal` needs the one-line `endlocal & set` trick (Section 16.1).
- Recursion depth is limited and slow; do not use it.
- A label inside a parenthesized block breaks `goto`/`call`; keep labels at top level.

Chapter 22. Engineering Patterns

22.1 Configuration files

A simple `key=value` file is parseable with `for /f`:

```
# settings.ini
retries=3
target=127.0.0.1
mode=audit
```

```
:load_config
rem usage: call :load_config "file"
if not exist "%~1" (echo ERROR: config not found: %~1 & exit /b 3)
for /f "usebackq eol=# tokens=1,* delims==" %%k in ("%~1") do (
    set "CFG_%%k=%%l"
)
exit /b 0
```

After this, `%CFG_retries%`, `%CFG_target%` and `%CFG_mode%` exist. Points:

- `eol=#` ignores comment lines; blank lines are skipped automatically.
- `tokens=1,*` puts everything after the first `=` into `%%l` so values may contain `=`.
- **Validate** every value before use (Section 22.2). A config file is input from outside your script.
- Prefix (`CFG_`) so a key can never overwrite a critical variable such as `PATH` or `COMSPEC`. Without a prefix, a line `PATH=...` in a config file silently changes your command search path.

22.2 Validation helpers

```
:is_number
rem usage: call :is_number "%VALUE%"   -> exit 0 if only digits
echo(%~1| findstr /r /x "[0-9][0-9]*" >nul
exit /b %ERRORLEVEL%

:is_safe_name
rem allowlist: letters, digits, dot, dash, underscore only
echo(%~1| findstr /r /x "[A-Za-z0-9._-][A-Za-z0-9._-]*" >nul
exit /b %ERRORLEVEL%
```

Allowlists beat blocklists. Trying to list every dangerous character (`& | < > ^ " % !`) will miss one; accepting only known-good characters will not. Part XX builds on this.

> **DANGER / CAUTION.** `echo(%~1| findstr ...` still places the value on a command line before validation, so a value containing `&` or `|` can break the line **before** your check runs. Use these helpers on values from files or from `set /p` read into a variable and expanded with delayed expansion (`echo(!VAL!| findstr ...`), never on raw, unquoted arguments from an untrusted party.

22.3 Backup, change, verify, rollback

The pattern for any modification:

```
1. validate inputs
2. back up current state to a known location
3. apply the change
4. verify the new state
5. if verification fails, restore the backup and exit nonzero
6. record what happened
```

Safe worked example using a disposable registry key (HKCU, no elevation needed):

```
@echo off
setlocal EnableExtensions
set "KEY=HKCU\Software\CmdLab"
set "BAK=%TEMP%\cmdlab_key.reg"

reg add "%KEY%" /v Mode /t REG_SZ /d original /f >nul || exit /b 1
reg export "%KEY%" "%BAK%" /y >nul || (echo backup failed & exit /b 1)

reg add "%KEY%" /v Mode /t REG_SZ /d changed /f >nul || goto :rollback
reg query "%KEY%" /v Mode | find "changed" >nul || goto :rollback
echo change verified
goto :done

:rollback
echo verification failed - restoring
reg import "%BAK%" >nul
exit /b 1

:done
reg query "%KEY%" /v Mode
reg delete "%KEY%" /f >nul
del "%BAK%" 2>nul
exit /b 0
```

The same shape works for files (`copy` to `.bak`, change, compare, move back) and for service configuration (`sc qc` output saved before `sc config`).

22.4 Temporary directories

```
set "WORK=%TEMP%\%APP%_%RANDOM%%RANDOM%"
md "%WORK%" || exit /b 1
rem ... use "%WORK%" ...
rd /s /q "%WORK%"
```

Never delete a path built from a variable without confirming it is non-empty and inside the expected parent:

```
if not defined WORK exit /b 1
echo "%WORK%" | findstr /i /c:"%TEMP%" >nul || (echo refusing to delete outside TEMP & exit /b 1)
```

22.5 Stale-lock detection

Record the start time and a process hint in the lock, and let a new instance reclaim an old lock:

```
:lock_acquire
set "LOCKDIR=%TEMP%\%APP%.lock"
mkdir "%LOCKDIR%" 2>nul && (>"%LOCKDIR%\owner.txt" echo %DATE% %TIME% & exit /b 0)
rem lock exists: how old is it? (more than 60 minutes -> consider stale)
forfiles /p "%LOCKDIR%" /m owner.txt /d -1 >nul 2>&1 && (
    echo stale lock older than one day: removing
    rd /s /q "%LOCKDIR%" & mkdir "%LOCKDIR%" 2>nul && exit /b 0
)
echo another instance is running
exit /b 1
```

`forfiles /d -1` matches files modified more than one day ago. Finer-grained checks need PowerShell. Any lock scheme in `%TEMP%` is advisory only.

22.6 Structured output

Machines want predictable formats. Emit CSV with a header and quote every field:

```
>"%OUT%" echo "timestamp","host","item","status"
>>"%OUT%" echo "%DATE% %TIME%","%COMPUTERNAME%","disk C:","ok"
```

For anything with nested data, call PowerShell (`ConvertTo-Json`) instead of hand-building JSON in batch; escaping rules make hand-built JSON fragile.

22.7 Timestamps

`%DATE%` and `%TIME%` depend on regional settings, so file names built from them break across machines. Locale-proof options:

```
for /f %%t in ('powershell -NoProfile -Command "Get-Date -Format yyyyMMdd-HHmmss"') do set "STAMP=%%t"
echo %STAMP%
```

This costs one PowerShell start (about a few hundred milliseconds); call it once per run, not once per log line. (The older `wmic os get localdatetime` approach should not be used in new scripts because WMIC is deprecated.)

22.8 Colored output and progress

Windows 10 and later consoles understand ANSI escape sequences. Build an ESC character once:

```
for /f %%e in ('echo prompt $E^| cmd') do set "ESC=%%e"
echo %ESC%[32mOK%ESC%[0m  %ESC%[31mFAILED%ESC%[0m
```

Progress that overwrites one line uses a carriage return, which batch cannot print directly; the common approach is simply to print steps (`[3/10] checking...`). Fancy animation is a sign the job belongs in PowerShell. Never color-code as the **only** signal; redirect to a file and the codes become garbage. Check `if "%NO_COLOR%"==""` before emitting escapes, and drop them when output is redirected.

22.9 Administrator detection (and why it is only a convenience)

```
:require_admin
fltmc >nul 2>&1 && exit /b 0
echo ERROR: run this from an elevated prompt.
exit /b 4
```

Better design: tell the user what to do, but **do not auto-elevate** from inside a batch script (tricks that relaunch via other interpreters are indistinguishable from abuse in telemetry and are hard to make safe).

22.10 Self-diagnostics

A tool that can explain its own environment saves hours of support:

```
:diag
echo --- diagnostics ---
echo script   : %~f0
echo version  : %VERSION%
echo user     : %USERDOMAIN%\%USERNAME%
echo host     : %COMPUTERNAME%
echo cwd      : %CD%
ver
where cmd
echo cmd ext  : %CMDEXTVERSION%
whoami /groups | find "Mandatory Label"
exit /b 0
```

Add a `/diag` switch to your parser and ask users to attach its output to bug reports.

Lab 66. Build and test the skeleton

Objective. Run the Section 21.3 skeleton through every path. **Difficulty.** [A]. **Prerequisites.** Chapter 21. **Environment.** Scratch folder, standard prompt.

Commands.

```
skeleton /?
skeleton /bogus
echo %ERRORLEVEL%
skeleton
skeleton /go /verbose /out "%TEMP%\cmdlab\out"
type "%TEMP%\cmdlab\out\skeleton.log"
```

Expected result. Usage with code 0; error plus usage with code 2; dry-run message; a real run that logs `doing the real work here`; the log file contains the timestamped lines. **Explanation.** Each path ends at `:end`, which releases the lock. **Verification.** `dir "%TEMP%\skeleton.lock"` should report not found after every run. **Troubleshooting.** If the lock folder persists, a run was killed mid-way: `rd "%TEMP%\skeleton.lock"`. **Cleanup.** `del skeleton.cmd & rd /s /q out`. **Challenge.** Add `/diag`. **Solution.** Add a `:parse` line calling `:diag` then `goto :end` with RC 0, using the function in Section 22.10.

Lab 67. Config file loader with allowlist validation

Objective. Load settings safely. **Difficulty.** [A]. **Prerequisites.** Sections 22.1-22.2. **Environment.** Scratch folder.

Commands. Create `settings.ini` (three lines from Section 22.1), a script `l67.cmd` containing the loader plus:

```
call :load_config "%~dp0settings.ini" || exit /b 3
set "VAL=%CFG_retries%"
call :is_number "!VAL!" || (echo retries must be a number & exit /b 2)
echo retries=%CFG_retries% target=%CFG_target% mode=%CFG_mode%
```

(enable delayed expansion at the top), then run it, edit `retries=abc` and run again.

Expected result. First run prints the three values; second run reports the validation error with code 2. **Explanation.** Values from files are untrusted input. **Verification.** `echo %ERRORLEVEL%`. **Troubleshooting.** Ensure the # comment line starts in column 1. **Cleanup.** `del l67.cmd settings.ini`. **Challenge.** Reject `mode` values other than `audit` or `fix`. **Solution.** `if /i not "%CFG_mode%"=="audit" if /i not "%CFG_mode%"=="fix" exit /b 2`.

Lab 68. Backup, change, verify, rollback

Objective. Execute the Section 22.3 pattern and force a rollback. **Difficulty.** [A]. **Prerequisites.** Section 22.3. **Environment. VM** or personal machine; uses only `HKCU\Software\CmdLab`.

Commands. Save the example as `l68.cmd` and run it. Then change the verification line to look for `changedX` (which will fail) and run again.

Expected result. First run: `change verified`, final query shows `changed`, key removed. Second run: `verification failed - restoring`, exit code 1, `Mode` back to `original`. **Explanation.** Verification failure triggers the rollback path. **Verification.** After the second run `reg query HKCU\Software\CmdLab` shows `original`. **Troubleshooting.** `reg import` fails if the export was empty; confirm the `.reg` file exists. **Cleanup.** `reg delete HKCU\Software\CmdLab /f & del %TEMP%\cmdlab_key.reg & del l68.cmd`. **Challenge.** Apply the same pattern to a file. **Solution.** `copy /y f.txt f.txt.bak`, edit, compare with `fc`, `copy /y f.txt.bak f.txt` on mismatch.

Lab 69. Stale-lock handling

Objective. Prove the lock blocks a second instance. **Difficulty.** [I]. **Prerequisites.** Section 21.3. **Environment.** Two prompts.

Commands. In prompt A run a copy of the skeleton with `pause` inserted at the start of `:main`; in prompt B run `skeleton`.

Expected result. Prompt B reports another instance is running and exits with 9. After pressing a key in A and finishing, B succeeds. **Explanation.** `mkdir` fails if the folder exists, atomically. **Verification.** Exit code in B. **Troubleshooting.** A leftover lock from a killed run needs manual removal. **Cleanup.** `rd "%TEMP%\skeleton.lock" 2>nul`. **Challenge.** Show the lock owner time in the error. **Solution.** `type "%TEMP%\skeleton.lock\owner.txt"` after storing it as in Section 22.5.

Lab 70. Colored status output with a safe fallback

Objective. Emit ANSI colors only when appropriate. **Difficulty.** [I]. **Prerequisites.** Section 22.8. **Environment.** Windows Terminal or Windows 10+ console.

Commands. Save as `l70.cmd`:

```
@echo off
setlocal EnableExtensions
set "ESC="
if not defined NO_COLOR for /f %%e in ('echo prompt $E^| cmd') do set "ESC=%%e"
echo %ESC%[32mPASS%ESC%[0m disk space
echo %ESC%[31mFAIL%ESC%[0m service stopped
```

Run it; then `set NO_COLOR=1` and run again; then `l70 > out.txt` and `type out.txt`.

Expected result. Colored words; plain text when `NO_COLOR` is set; escape codes appear in the redirected file when color was on. **Explanation.** ANSI codes are just bytes. **Verification.** `certutil -dump out.txt`. **Troubleshooting.** Old consoles print `?[32m`; use Windows Terminal. **Cleanup.** `set NO_COLOR=` and `del l70.cmd out.txt`. **Challenge.** Detect redirected output. **Solution.** There is no reliable pure-CMD test; accept a `/nocolor` switch instead.

Part VII - The Windows Filesystem

NTFS structure, permissions (ACLs), ownership, links, streams, attributes, compression, encryption concepts and the commands that inspect and change them.

Chapter 23. NTFS Concepts

23.1 Drives, volumes, paths

A **disk** is hardware. A **partition** is a slice of a disk. A **volume** is a formatted partition (or spanned set) with a file system. Windows exposes a volume as a **drive letter** (`C:`), as a **mount point** (an empty folder acting as the volume), or by a **volume GUID path** (`\\?\Volume{...}\`). `mountvol` lists these; `fsutil fsinfo drives` lists letters.

Path forms:

Form	Example	Notes
Drive-absolute	`C:\Users\Alice`	normal
Drive-relative	`C:notes.txt`	relative to C:'s remembered folder (surprising)
Relative	`..\data`	relative to the current directory
UNC	`\\server\share\dir`	network
Extended-length	`\\?\C:\very\long\path`	bypasses the 260-character `MAX_PATH` limit for APIs that support it; many tools do not

23.2 NTFS building blocks

- The **Master File Table (MFT)** has one record per file or folder. A record stores **attributes**: standard information (timestamps, flags), file name(s), security descriptor reference, and data. Small files can live entirely inside the MFT record.
- A file can have multiple `$DATA` **streams**: the unnamed default plus **named alternate data streams** (ADS).
- Metadata files (`$MFT`, `$LogFile`, `$Bitmap`, `$Secure`, `$UsnJrnl`...) are hidden system structures.
- NTFS keeps an **MFT-level journal** for crash consistency (metadata) and an optional **USN change journal** recording file changes (used by search, backup and forensic tools).
- Every file has up to four timestamps (created, modified, accessed, MFT-changed); last-access updating is often disabled for performance (`fsutil behavior query disablelastaccess`).
- **Short (8.3) names** may exist beside long names (`dir /x`); creation can be disabled per volume.
- ReFS, exFAT and FAT32 are different: FAT variants have **no ACLs**, no hard links, no compression or EFS.

23.3 Attributes

Attribute	Meaning	Set with
Read-only (R)	Blocks normal writes/deletes	`attrib +r`
Hidden (H)	Hidden from default listings; **not security**	`attrib +h`
System (S)	OS file flag	`attrib +s`
Archive (A)	"Changed since last backup" marker	`attrib +a`
Compressed	NTFS compression on	`compact /c`

Attribute	Meaning	Set with
Encrypted	EFS on	`cipher /e`
Reparse point	Link/junction/other special	`mklink`, `fsutil reparsepoint`
Offline / Not content indexed / Integrity	Special flags	various

23.4 Links and reparse points

Type	Points to	Cross-volume	Created with
Hard link	Same file data (another name for the same MFT record)	No	`mklink /h`
Directory junction	A folder on a local volume	Yes (local)	`mklink /j`
Symbolic link (file or dir)	A path (may be relative, may be network)	Yes	`mklink` / `mklink /d`
Mount point	Volume root	n/a	`mountvol`
Other reparse points	OneDrive placeholders, deduplication, WSL	n/a	system

A **reparse point** is a file or folder carrying a tag and data that tells the file system driver "ask someone else." Links are one use. Deleting a hard link removes one name; the data goes only when the last name is gone. Deleting a junction or symlink removes the link, not the target.

`dir /a:l` lists reparse points; `fsutil reparsepoint query <path>` shows the tag and target.

23.5 Alternate data streams (concept)

An ADS is extra data attached to a file name, invisible to a plain `dir`. Windows itself uses one called `Zone.Identifier` to record that a file came from the internet (the "Mark of the Web"). Harmless demonstration:

```
echo visible> note.txt
echo hidden data> note.txt:extra
dir note.txt
dir /r note.txt
more < note.txt:extra
```

`dir` shows the size of the main stream only; `dir /r` also lists `note.txt:extra:$DATA`. Copying the file to FAT or through some transfer methods **drops** the streams. Defenders check streams (`dir /r`, `streams.exe` from Sysinternals) because data stashed there escapes casual inspection and many size-based checks. Zone.Identifier streams are *evidence*: they record origin (ZoneId 3 = Internet).

23.6 Compression, encryption, quotas

- **NTFS compression** (`compact`) transparently compresses file data; it is per-file or per-folder and has CPU cost. It is unrelated to ZIP.
- **EFS (Encrypting File System)** (`cipher /e`) encrypts file contents with a per-file key protected by the user's certificate. It is **not available on Home editions**, it protects against offline access, not against the logged-in user or malware running as that user, and **losing the certificate means losing the data**.

BitLocker (`manage-bde`, Part XVII) encrypts whole volumes and is the primary disk-encryption answer.

- **Disk quotas** (`fsutil quota`) limit per-user use of a volume.
- `cipher /w:folder` overwrites **free space**, which reduces recoverability of deleted data on classic disks; on SSDs, wear leveling makes overwrite guarantees weak. Full-disk encryption from day one is the real control.

23.7 SIDs, ACLs and how access is decided

- A **SID (security identifier)** is the permanent internal name of a user, group or service account, for example `S-1-5-21-...-1001`. Names are lookups; **access control uses SIDs.**
- Well-known SIDs: `S-1-1-0` Everyone, `S-1-5-18` SYSTEM, `S-1-5-32-544` Administrators, `S-1-5-32-545` Users, `S-1-5-11` Authenticated Users.
- Every securable object has a **security descriptor**: **owner** SID, **DACL** (discretionary ACL: who may do what), **SACL** (system ACL: what to audit, and integrity labels).
- A DACL is an ordered list of **ACEs** (access control entries): each is *allow* or *deny*, for a SID, with rights and inheritance flags.
- Evaluation: Windows walks the DACL **in order** comparing the requesting token's SIDs. By convention explicit denies are placed before allows, so **deny wins** over allow. A missing DACL (NULL) means everyone has full access; an *empty* DACL means nobody does.
- The **owner** can always change the DACL (READ_CONTROL/WRITE_DAC implicit right), which is why `takeown` is the first step when access is lost.
- **Inheritance:** ACEs flagged inheritable flow from parent to children. `(OI)` object inherit (files), `(CI)` container inherit (folders), `(IO)` inherit-only (applies to children, not this folder), `(NP)` no-propagate. A child can **block inheritance**, converting inherited entries to explicit copies.

> **NOTE.** Administrators do not automatically have access to every file. They have the **right to take ownership** (SeTakeOwnershipPrivilege) and then grant themselves access. Ownership changes are visible in audit logs and are a standard investigation signal.

Chapter 24. Permissions and Filesystem Command Cards

ICACLS [External, ACLs]

Field	Detail
Purpose / Why	Displays and modifies DACLs; backs up and restores them; resets inheritance
Syntax	`icacls path` ; `icacls path /grant[:r] principal:perm` ; `/deny` ; `/remove[:g|:d] principal` ; `/inheritance:e|d|r` ; `/save file [/t]` ; `/restore dir` ; `/reset` ; `/setowner name` ; `/t` recurse, `/c` continue on errors, `/q` quiet
Key switches	Rights: `F` full, `M` modify, `RX` read and execute, `R` read, `W` write, `D` delete. Inheritance: `(OI)(CI)(IO)(NP)`. `/inheritance:r` removes inherited ACEs, `:d` converts them to explicit, `:e` re-enables
Beginner	`icacls note.txt`
Intermediate	`icacls labdir /grant "%USERNAME%:(OI)(CI)R"`
Advanced	`icacls labdir /save acl_backup.txt /t` then later `icacls . /restore acl_backup.txt`
Real-world	Giving a service account modify rights on a data folder without touching anything else
Output	One line per ACE: `DOMAIN\name:(I)(OI)(CI)(F)`. `(I)` means inherited. Ends with `Successfully processed 1 files; Failed processing 0 files`.
Exit / Errors	0 success; 5 or `Access is denied.` when you lack WRITE_DAC; `No mapping between account names and security IDs was done.` for unknown names
Admin	Not for your own objects; yes for others'
Versions	Present since Vista; replaces `cacls` and `xcacls`
Mistakes	`/grant` **adds** rights while `/grant:r` **replaces** that principal's existing ones. `/inheritance:r` plus a mistake can lock everyone out. Using `/t` on `C:\` or `C:\Windows` can break the OS.
Security	Check for `Everyone:(F)` or `Users:(M)` on program folders and service binaries; these allow replacement of trusted code (Part XIX)
Lab / Cleanup	Lab 28; `icacls /reset` only inside the lab folder
Related	`takeown`, `cacls` (deprecated), `fsutil`, `attrib`
PowerShell	`Get-Acl`, `Set-Acl`, `New-Object System.Security.AccessControl.FileSystemAccessRule`

DANGER / CAUTION. Always take a backup before changing ACLs in bulk: `icacls "target" /save "%USERPROFILE%\acl_backup.txt" /t /c`. Restore with `icacls "parent-of-target" /restore "%USERPROFILE%\acl_backup.txt"`. Never run recursive ACL changes on system folders.

TAKEOWN [External, ownership]

Field	Detail
Purpose / Why	Makes you (or the Administrators group) the owner of a file or folder so you can then fix its ACL
Syntax	`takeown /f path [/a] [/r [/d y|n]] [/skipsl]`
Key switches	`/a` give ownership to Administrators group rather than the current user, `/r` recurse, `/d y` answer yes to the "no permission to list" prompt
Beginner	`takeown /f locked.txt`
Intermediate	`takeown /f labdir /r /d y`
Advanced	`takeown /f labdir /a /r /d y` then `icacls labdir /grant Administrators:(OI)(CI)F`
Real-world	Recovering files left over from a deleted user profile
Output	`SUCCESS: The file (or folder): "path" now owned by user "NAME"`
Exit / Errors	`ERROR: Access is denied` when not elevated
Admin	Yes (needs SeTakeOwnershipPrivilege, held by Administrators)
Versions	Same on 10 and 11
Mistakes	Taking ownership of system files (such as TrustedInstaller-owned ones) and breaking servicing; ownership of OS files should not be changed without a rollback plan
Security	Ownership changes are a classic access-escalation step by someone with admin rights; audit with event 4670 (permissions changed) if object-access auditing is on
Lab / Cleanup	Lab 28 in a lab folder only
Related	`icacls /setowner`, `whoami /priv`
PowerShell	`(Get-Acl).SetOwner(...)`, `Set-Acl`

FSUTIL [External, file system utility]

Field	Detail
Purpose / Why	Low-level NTFS and volume information and some tuning
Syntax	`fsutil <area> <subcommand> ...` ; areas include `fsinfo`, `volume`, `file`, `hardlink`, `reparsepoint`, `usn`, `quota`, `behavior`, `sparse`
Key switches	`fsutil fsinfo drives`, `fsutil fsinfo volumeinfo C:`, `fsutil volume diskfree C:`, `fsutil hardlink list file`, `fsutil reparsepoint query path`, `fsutil behavior query disablelastaccess`
Beginner	`fsutil fsinfo drives`
Intermediate	`fsutil volume diskfree C:`
Advanced	`fsutil file layout path` (MFT record details); `fsutil usn queryjournal C:`
Real-world	Finding every name of a hard-linked file; checking whether 8.3 names or last-access updates are enabled

Field	Detail
Output	Key/value lines; sizes in bytes (hexadecimal in some outputs)
Exit / Errors	`Error: Access is denied.` or `The FSUTIL utility requires that you have administrative privileges.`
Admin	Yes for most subcommands
Versions	Subcommand set grows with Windows builds; `fsutil /?` shows what your build supports
Mistakes	Using `fsutil behavior set`, `fsutil usn deletejournal`, `fsutil dirty set` or `fsutil file setzerodata` casually. Some of these destroy forensic or recovery data
Security	The USN journal is forensic evidence of file creation, rename and deletion; deleting it destroys history
Lab / Cleanup	Read-only subcommands only (Lab 31)
Related	`mountvol`, `diskpart`, `chkdsk`, `icacls`
PowerShell	`Get-Volume`, `Get-Disk`, `Get-Partition`, `Get-Item -Stream *`

CIPHER and COMPACT [External, encryption and compression]

Field	Detail
Purpose / Why	`cipher`: show or set EFS encryption and wipe free space. `compact`: show or set NTFS compression.
Syntax	`cipher [/e|/d] [/s:dir] [path]` ; `cipher /w:dir` ; `cipher /u /n` (list encrypted files). `compact [/c|/u] [/s] [path]`
Key switches	`cipher /e` encrypt, `/d` decrypt, `/w:dir` wipe unused disk space under that volume, `/s:dir` recurse. `compact /c` compress, `/u` uncompress, `/s` subfolders, `/q` summary
Beginner	`cipher labdir` (shows `U` unencrypted / `E` encrypted); `compact labdir`
Intermediate	`compact /c /s:labdir` then `compact /s:labdir` to see the ratio
Advanced	`compact /compactos:query` shows whether Compact OS is active
Real-world	Compressing old log folders; wiping free space before returning a disk
Output	Status per file; compression ratio such as `1.8 to 1`
Exit / Errors	`cipher`: EFS not supported on this edition or file system (Home, FAT)
Admin	No for own files
Versions	EFS needs Pro/Enterprise/Education, not Home; compression needs NTFS; `compact` of system files can break servicing
Mistakes	Encrypting without backing up the EFS certificate and key (`cipher /x`); compressing already-compressed media (no gain); using `cipher /w` on a drive with little free space for hours on an SSD
Security	EFS protects against another account and offline theft but not against the logged-in user. Back up the certificate to a safe place
Lab / Cleanup	Lab 36 uses a disposable folder
Related	`manage-bde`, `attrib`, `fsutil sparse`
PowerShell	`[System.IO.File]::Encrypt()`, `Compress-Archive` for ZIP

ROBOCOPY [External, robust copy]

Field	Detail
Purpose / Why	Reliable copy, sync and mirror of folder trees with retry, logging, timestamps and ACLs. Replaces `xcopy` for serious work.
Syntax	`robocopy source dest [files] [options]`
Key switches	`/e` subfolders incl. empty, `/s` non-empty subfolders, `/mir` mirror (**deletes extras in dest**), `/purge`, `/r:n /w:s` retries and wait, `/z` restartable, `/mt:n` multithread, `/copy:DATSOU` or `/copyall` or `/sec`, `/dcopy:DAT`, `/xd`, `/xf` exclusions, `/log:file`, `/tee`, `/np`, `/l` **list only (dry run)**, `/xo`, `/fft`
Beginner	`robocopy src dst /e`
Intermediate	`robocopy src dst /e /r:2 /w:2 /log:copy.log /np`
Advanced	`robocopy src dst /mir /l /log:preview.log` (preview, then rerun without `/l` once reviewed)
Real-world	Nightly backup that keeps ACLs: `robocopy D:\data E:\backup /e /copy:DATSOU /dcopy:DAT /r:3 /w:5 /mt:8 /log+:backup.log /np`
Output	Summary table: Dirs, Files, Bytes, Times, Speed with Total/Copied/Skipped/Mismatch/FAILED/Extras columns
Exit / Errors	Bit flags: **0** nothing to do, **1** files copied, **2** extra files in dest, **4** mismatches, **8** some copies failed, **16** fatal error. Values combine (3 = copied plus extras). **Treat 8 or higher as failure.**
Admin	Only for `/b` backup mode or protected sources and `/sec` of some objects
Versions	Built in since Vista; very old switches vary
Mistakes	`/mir` with source and destination swapped erases the real data; always run `/l` first. Treating exit code 1 as an error. Trailing backslash and quote bug (`"C:\dir\"`).
Security	`robocopy /copyall` preserves owner and audit info and can reveal data you did not mean to copy; logs list file names
Lab / Cleanup	Lab 38
Related	`xcopy`, `copy`, `fc`, `icacls /save`
PowerShell	`Copy-Item -Recurse`, still often calls `robocopy`

DANGER / CAUTION. `robocopy ... /mir` and `/purge` **delete files in the destination**. Rehearse with `/l` and read the `*EXTRA File` lines; a typo in the destination path can wipe an unrelated folder.

Lab 26. Reading a path with ADS and attributes

Objective. Compare what `dir` shows with and without `/r`, `/x`, `/q`. **Difficulty.** [B]. **Prerequisites.** Chapter 23. **Environment.** Scratch folder, NTFS.

Commands.

```
echo visible> note.txt
echo hidden data> note.txt:extra
dir note.txt
dir /r note.txt
dir /q note.txt
dir /x
```

Expected result. `dir` shows `note.txt` with a size of 9 (`visible` plus CRLF); `dir /r` additionally lists `note.txt:extra:$DATA`; `/q` shows the owner; `/x` shows short names where they exist. **Explanation.** The size counts only the default stream. **Verification.** `more < note.txt:extra`. **Troubleshooting.** On FAT/exFAT volumes the `:extra` write fails. **Cleanup.** `del note.txt`. **Challenge.** Delete only the extra stream. **Solution.** In CMD there is no direct command; use PowerShell `Remove-Item note.txt -Stream extra`, or delete the whole file.

Lab 27. Read ACLs on three different places

Objective. Learn to read `icacls` output. **Difficulty.** [B]. **Prerequisites.** Section 23.7. **Environment.** Standard prompt.

Commands.

```
icacls "%USERPROFILE%"
icacls "%ProgramFiles%"
icacls "%WINDIR%\System32\cmd.exe"
```

Expected result. Your profile grants you full control and SYSTEM/Administrators; Program Files grants Users only read/execute (`(RX)`), with TrustedInstaller/Administrators/SYSTEM full; `cmd.exe` is owned by TrustedInstaller. **Explanation.** Programs live where ordinary users cannot write. **Verification.** `icacls` prints `(I)` for inherited entries. **Troubleshooting.** Names are localized on non-English Windows. **Cleanup.** None. **Challenge.** Find a folder under `C:\` that grants `Authenticated Users:(M)`. **Solution.** `icacls C:\* 2>nul | findstr /i "Authenticated Users:(M"` (results vary; investigate each).

Lab 28. Change ACLs and ownership in a sandbox folder

Objective. Grant, deny, inherit and restore. **Difficulty.** [I]. **Prerequisites.** Lab 27. **Environment. VM**, elevated prompt, scratch folder, NTFS.

Commands.

```
md acl_lab & echo x> acl_lab\f.txt
icacls acl_lab /save acl_before.txt /t /c
icacls acl_lab /inheritance:d
icacls acl_lab /remove "Users"
icacls acl_lab /grant "%USERNAME%:(OI)(CI)R"
icacls acl_lab
icacls acl_lab\f.txt
takeown /f acl_lab /r /d y
icacls acl_lab /reset /t
icacls acl_lab
```

Expected result. After `/inheritance:d` the inherited entries become explicit; removing `Users` drops that line; the grant adds you with read; `f.txt` shows the same entries inherited from the folder; `/reset` returns to

inherited defaults. **Explanation.** Inheritance can be disabled (copying parent entries) or reset. **Verification.** Compare final `icacls` output with `acl_before.txt`. **Troubleshooting.** If `icacls /reset` complains about access, run `takeown` first. **Cleanup.** `icacls . /restore acl_before.txt` is not needed; just `rd /s /q acl_lab & del acl_before.txt`. **Challenge.** Deny yourself write on `f.txt`, confirm `echo y>> acl_lab\f.txt` fails, then remove the deny. **Solution.** `icacls acl_lab\f.txt /deny "%USERNAME%:W"`, test, then `icacls acl_lab\f.txt /remove:d "%USERNAME%"`.

Lab 29. Hard links, junctions and symlinks

Objective. See the three link types behave differently. **Difficulty.** [I]. **Prerequisites.** Section 23.4. **Environment.** Scratch folder; elevation or Developer Mode for symlinks.

Commands.

```
echo original> orig.txt
mklink /h hard.txt orig.txt
md realdir & echo inside> realdir\in.txt
mklink /j junc realdir
mklink /d symd realdir
dir /a:l
echo changed>> orig.txt
type hard.txt
fsutil hardlink list orig.txt
del orig.txt
type hard.txt
rd junc
dir realdir
```

Expected result. `dir /a:l` lists the junction and symlink with targets; `hard.txt` shows both lines after the append and still works after `orig.txt` is deleted; `rd junc` removes the link while `realdir\in.txt` remains. **Explanation.** A hard link is another name for the same data; junction and symlink are redirects. **Verification.** `fsutil reparsepoint query symd`. **Troubleshooting.** `You do not have sufficient privilege` for symlinks means elevate or enable Developer Mode. **Cleanup.** `rd symd & rd /s /q realdir & del hard.txt`. **Challenge.** Show that a hard link cannot be made to a folder. **Solution.** `mklink /h hl realdir` fails with `Access is denied.`

Lab 30. Find every link in a tree (audit)

Objective. Locate reparse points that could redirect file operations. **Difficulty.** [I]. **Prerequisites.** Lab 29. **Environment.** Standard prompt.

Commands.

```
dir /s /a:l "%USERPROFILE%" 2>nul
dir /s /a:l "%ProgramData%" 2>nul
```

Expected result. A list of junctions and symlinks (OneDrive and AppData compatibility junctions are common). **Explanation.** Writable locations containing links deserve a look because privileged programs that follow links can be tricked into writing elsewhere. **Verification.** `fsutil reparsepoint query <path>` for any entry. **Troubleshooting.** Slow on large profiles; narrow the path. **Cleanup.** None. **Challenge.** Count them. **Solution.** `dir /s /a:l /b "%USERPROFILE%" 2>nul | find /c /v ""`.

Lab 31. Read-only fsutil survey

Objective. Gather volume facts safely. **Difficulty.** [I]. **Prerequisites.** `fsutil` card. **Environment.** Elevated prompt, **VM**.

Commands.

```
fsutil fsinfo drives
fsutil fsinfo volumeinfo C:
fsutil volume diskfree C:
fsutil behavior query disablelastaccess
fsutil behavior query disable8dot3
fsutil usn queryjournal C:
```

Expected result. Drive list, file-system name and flags (such as `Supports Reparse Points`), free bytes, and the two behavior settings. **Explanation.** All are read-only queries. **Verification.** Confirm `File System Name : NTFS`. **Troubleshooting.** Without elevation each command fails. **Cleanup.** None. **Challenge.** Compute free GB. **Solution.** `set /a` cannot exceed 32 bits; use PowerShell: `(Get-Volume C).SizeRemaining/1GB`.

Lab 32. Attributes as evidence

Objective. Find hidden and system files in a folder. **Difficulty.** [B]. **Prerequisites.** `attrib` card. **Environment.** Scratch folder.

Commands.

```
echo a> v.txt & echo b> h.txt & echo c> s.txt
attrib +h h.txt & attrib +s +h s.txt
dir /b
dir /b /a
dir /b /a:h
dir /b /a:s
attrib
```

Expected result. Plain `dir /b` lists only `v.txt`; `/a` lists all three; filters show the right subsets; `attrib` prints the flag letters. **Explanation.** Hidden files are common in malware staging, and also in normal OS files. **Verification.** Counts 1, 3, 2, 1. **Troubleshooting.** `/a:s` on `s.txt` requires both flags removed later. **Cleanup.** `attrib -s -h s.txt & attrib -h h.txt & del v.txt h.txt s.txt`. **Challenge.** List hidden files recursively under your lab folder. **Solution.** `dir /s /b /a:h`.

Lab 33. Timestamps and last-write ordering

Objective. Read the four timestamps. **Difficulty.** [I]. **Prerequisites.** Section 23.2. **Environment.** Scratch folder.

Commands.

```
echo one> t.txt
dir /t:c t.txt
dir /t:w t.txt
dir /t:a t.txt
timeout /t 2 /nobreak >nul
echo two>> t.txt
dir /t:w t.txt
```

Expected result. Creation time stays fixed; last-write time moves after the append. **Explanation.** Copying a file gives the copy a new creation time but keeps its modified time; moving within a volume keeps all. **Verification.** `copy t.txt u.txt` then compare `dir /t:c`. **Troubleshooting.** Last-access times may not update (disabled by default on modern Windows). **Cleanup.** `del t.txt u.txt`. **Challenge.** Explain why a file's modified time can precede its created time. **Solution.** It was copied or extracted from an archive, which preserved the original modified time.

Lab 34. Missing-access recovery drill

Objective. Practice the safe order of operations when access is denied. **Difficulty.** [I]. **Prerequisites.** Lab 28. **Environment. VM**, elevated.

Commands.

```
md locked & echo x> locked\f.txt
icacls locked /inheritance:r
icacls locked /grant:r SYSTEM:(OI)(CI)F
dir locked
takeown /f locked /r /d y
icacls locked /grant "%USERNAME%:(OI)(CI)F" /t
dir locked
```

Expected result. After step 3 your account cannot list the folder (unless elevated as owner); after `takeown` and the grant it works. **Explanation.** Owner can always reset DACL; take ownership, then grant. **Verification.** `icacls locked` shows your account. **Troubleshooting.** If you are still denied, confirm the prompt is elevated. **Cleanup.** `rd /s /q locked`. **Challenge.** Do it without `/t`. **Solution.** Apply grant on the folder only; new children inherit.

Lab 35. Compact and measure

Objective. Compress text-heavy data and read the ratio. **Difficulty.** [B]. **Prerequisites.** `compact` card. **Environment.** Scratch folder, NTFS.

Commands.

```
md cdemo
dir /s /b %WINDIR%\System32 > cdemo\list.txt
for /l %i in (1,1,5) do @type cdemo\list.txt >> cdemo\big.txt
compact cdemo\big.txt
compact /c cdemo\big.txt
compact cdemo\big.txt
compact /u cdemo\big.txt
```

Expected result. Before: ratio `1.0 to 1`; after: better than `2 to 1` for repetitive text. **Explanation.** NTFS compresses repeated patterns very well. **Verification.** Compare `dir` size (logical) with compact's compressed size. **Troubleshooting.** Not supported on FAT. **Cleanup.** `rd /s /q cdemo`. **Challenge.** Compress a ZIP file and compare. **Solution.** Ratio near `1.0 to 1`; already compressed.

Lab 36. EFS concepts (Pro/Enterprise, VM only)

Objective. Observe encryption flags without risking real data. **Difficulty.** [I]. **Prerequisites.** Section 23.6. **Environment. VM** on Pro/Enterprise/Education with a snapshot; skip on Home.

Commands.

```
md efs_lab & echo secret> efs_lab\s.txt
cipher efs_lab
cipher /e efs_lab
cipher efs_lab
type efs_lab\s.txt
cipher /d efs_lab
```

Expected result. `U` before, `E` after; `type` still works for your account (transparent decryption). **Explanation.** EFS keys belong to your user certificate. **Verification.** `cipher efs_lab` prints `E` next to the file after encryption and `U` after decryption. **Troubleshooting.** `The request is not supported` on Home or FAT. **Cleanup.** `cipher /d efs_lab & rd /s /q efs_lab`. **Challenge.** Explain why you must back up the certificate. **Solution.** Losing the profile/key makes the data unrecoverable; export with `cipher /x` to a secure location.

Lab 37. Short names and long path limits

Objective. See 8.3 names and the path-length boundary. **Difficulty.** [I]. **Prerequisites.** Section 23.1. **Environment.** Scratch folder.

Commands.

```
md "a long folder name"
echo x> "a long folder name\a long file name.txt"
dir /x "a long folder name"
for %i in ("a long folder name\a long file name.txt") do @echo %~si
```

Expected result. A short alias such as `ALONGF~1`; `%~si` prints the short path. **Explanation.** Short names exist for legacy compatibility and may be disabled. **Verification.** `fsutil behavior query disable8dot3`. **Troubleshooting.** Short names are absent if disabled. **Cleanup.** `rd /s /q "a long folder name"`. **Challenge.** Explain the security relevance. **Solution.** Short names can bypass filters that only match long names; log and filter on resolved paths.

Lab 38. Safe backup with robocopy and exit-code handling

Objective. Back up with a dry run first, then verify. **Difficulty.** [I]. **Prerequisites.** `robocopy` card. **Environment.** Scratch folder.

Commands. Save as `l38.cmd`:

```
@echo off
setlocal EnableExtensions
set "SRC=%~dp0src"
set "DST=%~dp0dst"
if not exist "%SRC%\" md "%SRC%" & echo a> "%SRC%\a.txt" & echo b> "%SRC%\b.txt"
echo --- preview ---
robocopy "%SRC%" "%DST%" /e /l /njh /njs /ndl /nc /ns
echo --- copy ---
robocopy "%SRC%" "%DST%" /e /r:2 /w:1 /np /log:"%~dp0l38.log"
set "RC=%ERRORLEVEL%"
echo robocopy exit code: %RC%
if %RC% geq 8 (echo BACKUP FAILED & exit /b 1)
echo --- verify ---
fc "%SRC%\a.txt" "%DST%\a.txt" >nul && echo a.txt identical
exit /b 0
```

Run it twice.

Expected result. First run: code 1 (files copied); second run: code 0 (nothing to do). **Explanation.** Codes below 8 are success. **Verification.** `type l38.log`. **Troubleshooting.** A swapped SRC/DST with `/mir` would delete data; this lab uses `/e` only. **Cleanup.** `rd /s /q src dst & del l38.cmd l38.log`. **Challenge.** Switch to `/mir`, add a stray file in `dst`, preview with `/l`, observe `*EXTRA File`, then run for real. **Solution.** The stray file is deleted; this is why preview matters.

Lab 39. NTFS permission auditor mini-script

Objective. Flag broad write access in a folder tree. **Difficulty.** [A]. **Prerequisites.** Lab 27. **Environment.** Standard prompt, a test folder tree.

Commands. Save as `l39.cmd`:

```
@echo off
setlocal EnableExtensions
if "%~1"=="" (echo Usage: %~nx0 folder & exit /b 2)
set /a HITS=0
for /f "delims=" %%d in ('dir /s /b /a:d "%~1" 2^>nul') do (
    icacls "%%d" 2>nul | findstr /i /c:"Everyone:(" /c:"Users:(M" /c:"Users:(F" /c:"Authenticated Users:(M" /c:"A
        echo BROAD: %%d
        set /a HITS+=1
    )
)
echo Findings: %HITS%
exit /b 0
```

Create a test tree, grant `Everyone` modify on one folder (`icacls t\x /grant Everyone:(OI)(CI)M`), run `l39 t`.

Expected result. The grant is flagged. **Explanation.** This is a coarse text match; it is a teaching starter, not a replacement for proper tools. **Verification.** Remove the grant and the finding disappears. **Troubleshooting.** `%HITS%` prints the value at parse time inside blocks; the final echo is outside the loop so it is correct; use `!HITS!` inside blocks if needed. **Cleanup.** `rd /s /q t & del l39.cmd`. **Challenge.** Output CSV. **Solution.** Write `"path","principal","rights"` rows using `>>` to a file.

Lab 40. Filesystem capstone: the lost-access ticket

Objective. Diagnose and fix a folder users cannot open, using evidence. **Difficulty.** [A]. **Prerequisites.** Labs 27-34. **Environment. VM**, elevated.

Commands. Create the problem:

```
md ticket & echo data> ticket\report.txt
icacls ticket /inheritance:r
icacls ticket /grant:r SYSTEM:(OI)(CI)F
icacls ticket /setowner SYSTEM
```

Now diagnose as a technician without looking at the setup: run `dir ticket`, `icacls ticket`, `whoami /groups`, then fix and document each step.

Expected result. You discover inheritance removed, no entry for your account, owner SYSTEM. **Explanation.** Evidence first (`icacls`, owner via `dir /q`), then the minimal fix. **Verification.** `dir ticket` works; `icacls ticket` shows only what you intend. **Troubleshooting.** If `takeown` is refused, the prompt is not elevated. **Cleanup.** `rd /s /q ticket`. **Challenge.** Restore the original inherited ACL. **Solution.** `takeown /f ticket /r /d y`, `icacls ticket /grant "%USERNAME%:(OI)(CI)F"`, then `icacls ticket /reset /t`. Write down: symptom, cause, command, verification, rollback.

Part VIII - Windows Processes

What a process is, how to inspect it from CMD, and how to investigate one that looks wrong.

Chapter 25. Process Concepts and Inspection

25.1 What a process is

A **program** is a file on disk. A **process** is a running instance: an isolated container holding an address space (memory), a **token** (who it runs as), a table of **handles** (open files, registry keys, mutexes...), an environment block, a current directory, and one or more **threads** (the units the CPU actually schedules).

Term	Meaning
PID	Process ID, a number unique **while the process lives** (PIDs are reused after exit)
Parent / child	The process that created another; the child's parent PID (PPID) is recorded at creation
Image	The executable file the process was started from
Command line	The exact text passed to the process; programs parse it themselves (Chapter 12)
Session	Session 0 holds services; interactive logons get session 1 and up
Integrity level	Low, Medium, High, System label on the token
Handle	A process's reference to a kernel object

Important facts for investigators:

1. The **parent PID can refer to a process that has already exited** (and the PID may now belong to something else). Never trust a parent link without checking creation times.
2. A process name is only a label. `svchost.exe` **in the wrong folder is not svchost.** Verify the full image path, signer and parent.
3. Command lines frequently contain secrets and are logged by security tools; avoid putting passwords in them.

25.2 Tools available from CMD

Tool	Provides	Notes
`tasklist`	PID, name, session, memory; services; modules; filters	Built in
`taskkill`	Ends processes	Built in
`wmic process`	Command line, parent PID, path	**Deprecated**; disabled by default on Windows 11 24H2 and later
PowerShell `Get-CimInstance Win32_Process`	The modern replacement for WMIC process data	Call from CMD when needed
`typeperf`, `logman`	Performance counters	Part XVIII
Sysinternals Process Explorer / Process Monitor / Sysmon	Deep inspection; process tree; signatures; full telemetry	Separate downloads from Microsoft

Getting command line, parent and path without WMIC:

```
powershell -NoProfile -Command "Get-CimInstance Win32_Process | Select-Object ProcessId,ParentProcessId,Name,Exec
```

or as CSV for scripts:

```
powershell -NoProfile -Command "Get-CimInstance Win32_Process | Select-Object ProcessId,ParentProcessId,Name,Exec
```

TASKLIST [External, processes]

Field	Detail
Purpose / Why	Lists running processes with several detail levels
Syntax	`tasklist [/v] [/svc] [/m [module]] [/fi "filter"] [/fo table|list|csv] [/nh] [/s computer /u user]`
Key switches	`/svc` services hosted in each process, `/v` verbose (status, user, window title, CPU time), `/m` loaded modules/DLLs, `/fi` filter (`"imagename eq cmd.exe"`, `"pid eq 4"`, `"username eq NAME"`, `"memusage gt 100000"`), `/fo csv /nh` for scripts
Beginner	`tasklist`
Intermediate	`tasklist /fi "imagename eq svchost.exe" /svc`
Advanced	`tasklist /v /fo csv /nh | findstr /i "notepad"`
Real-world	Confirming that exactly one instance of an agent runs; finding what holds a DLL (`tasklist /m foo.dll`)
Output	Columns Image Name, PID, Session Name, Session#, Mem Usage. `Services` = session 0; `Console` = interactive.
Exit / Errors	0; `INFO: No tasks are running which match the specified criteria.` is still exit 0
Admin	Not for basic list; some details (other users' names, modules) need elevation
Versions	Same on 10 and 11
Mistakes	Parsing the table by column position (width varies); use `/fo csv`
Security	Shows names only; verify path and signer with other tools. Remote use (`/s`) needs rights and open firewall ports.
Lab / Cleanup	Labs 81-82
Related	`taskkill`, `wmic process`, `sc queryex`
PowerShell	`Get-Process`, `Get-CimInstance Win32_Process`

TASKKILL [External, process termination]

Field	Detail
Purpose / Why	Ends processes by PID, name or filter
Syntax	`taskkill [/pid n | /im name] [/f] [/t] [/fi "filter"]`
Key switches	`/pid` by id (safest), `/im` by name (kills all matches), `/f` force, `/t` also child processes

Field	Detail
Beginner	`taskkill /pid 1234`
Intermediate	`taskkill /im notepad.exe`
Advanced	`taskkill /fi "windowtitle eq LabWindow*" /t`
Real-world	Clearing a hung application; stopping a runaway script and its children
Output	`SUCCESS: Sent termination signal to the process with PID 1234.`
Exit / Errors	128 not found; `Access is denied.` for protected or other-user processes; `/f` is needed when the process ignores the polite close request
Admin	For other users' or system processes
Versions	Same
Mistakes	`/im` kills **every** matching process, including ones belonging to other sessions; `/f` loses unsaved data; killing `csrss.exe` or `wininit.exe` crashes the system
Security	Security tools may be targets for termination; unexpected `taskkill` against an agent is a notable event (tamper attempts are logged by good EDR)
Lab / Cleanup	Lab 82
Related	`tasklist`, `sc stop`, `shutdown`
PowerShell	`Stop-Process -Id`, `-Name`, `-Force`

SYSTEMINFO and HOSTNAME [External, system facts]

Field	Detail
Purpose / Why	Summarizes OS version, install date, boot time, hardware, hotfixes, network cards
Syntax	`systeminfo [/fo table|list|csv] [/s computer]`
Beginner	`systeminfo`
Intermediate	`systeminfo | findstr /b /c:"OS" /c:"System Boot Time" /c:"Total Physical Memory"`
Advanced	`systeminfo /fo csv > si.csv`
Real-world	First command on any troubleshooting ticket; first item in an evidence package
Output	Host name, OS name/version/build, original install date, boot time, manufacturer, model, processors, BIOS version, memory, hotfix list, network adapters. Takes several seconds.
Exit / Errors	0; slow when WMI is unhealthy
Admin	No
Versions	Fields vary; hotfix list may be incomplete (not every update appears as a hotfix)
Mistakes	Using it to prove patch level; use Windows Update history or `DISM /Online /Get-Packages`
Security	Contains an inventory valuable to an attacker; treat output as sensitive
Lab / Cleanup	Lab 81
Related	`ver`, `hostname`, `driverquery`, `wmic` (deprecated)

Field	Detail
PowerShell	`Get-ComputerInfo`

Chapter 26. Investigating a Process

26.1 The five questions

For any process that looks suspicious or unknown, answer in order:

1. **What** is it? Name, full image path, file hash, signer, version resource.
2. **Who** started it? Parent process (and verify the parent is the same process by creation time), user account, session, integrity level.
3. **How** was it started? Full command line, working directory.
4. **What is it doing?** Network connections (`netstat -ano`), loaded modules, files and registry keys (Process Monitor), child processes.
5. **Why is it there?** Service, scheduled task, Run key, user action, installer. Persistence leads to the cause.

26.2 Evidence commands

```
tasklist /v /fo list
tasklist /svc
netstat -ano | findstr " 1234"
where /r C:\Windows notepad.exe
certutil -hashfile "C:\Windows\System32\notepad.exe" SHA256
powershell -NoProfile -Command "Get-AuthenticodeSignature 'C:\Windows\System32\notepad.exe' | Format-List Status,
```

`certutil -hashfile` computes a cryptographic hash (algorithms include MD5, SHA1, SHA256, SHA512). Compare against a known-good source; do **not** treat "unsigned" as "malicious" or "signed" as "safe": many legitimate tools are unsigned and attackers abuse signed binaries (living off the land). Judgment needs context.

26.3 Expected parent/child relationships (Windows 10/11)

Process	Expected parent	Suspicious if
`smss.exe`	System (PID 4)	any other parent
`wininit.exe`, `winlogon.exe`	`smss.exe` (exited)	has a live parent that is not smss
`services.exe`	`wininit.exe`	other parent
`svchost.exe`	`services.exe`	other parent, wrong path, no `-k` argument
`lsass.exe`	`wininit.exe`	duplicated, or in a path other than `System32`
`explorer.exe`	(userinit exited)	multiple per user session without reason
`cmd.exe`	`explorer.exe`, a terminal, another script host	spawned by an Office app, a PDF reader or a web browser without clear user action

Treat unusual lineage as a lead, not a verdict. Installers, remote management tools and developer tools legitimately launch shells from odd parents. Always corroborate.

26.4 What a benign chain looks like

```
explorer.exe
  +-- WindowsTerminal.exe
        +-- cmd.exe                 <- you typed here
              +-- tasklist.exe      <- short-lived child
```

Short-lived children of `cmd.exe` appear and disappear in seconds, so logging them requires process-creation auditing (event 4688 with command-line capture enabled by policy, or Sysmon event 1), not a snapshot with `tasklist`.

Lab 81. Baseline a clean machine

Objective. Capture process, service and system facts as a reference. **Difficulty.** [B]. **Prerequisites.** Chapter 25. **Environment. VM**, scratch folder.

Commands.

```
systeminfo > base_sys.txt
tasklist /v /fo csv > base_tasks.csv
tasklist /svc /fo csv > base_svc.csv
for /f %f in ('dir /b base_*') do @echo %f && find /c /v "" %f
```

Expected result. Three files with plausible line counts (tens to a hundred processes). **Explanation.** A baseline lets you spot new items later with `fc`. **Verification.** Open the CSV in a spreadsheet. **Troubleshooting.** `systeminfo` can take 20 seconds. **Cleanup.** Keep the files for Lab 83. **Challenge.** Add a hash of every running executable. **Solution.** Use PowerShell `Get-Process | Select -Unique Path | Get-FileHash`.

Lab 82. Start, find, and stop a harmless process

Objective. Use `tasklist` filters and `taskkill` precisely. **Difficulty.** [B]. **Prerequisites.** Lab 81. **Environment.** Any prompt.

Commands.

```
start "LabWindow" notepad.exe
tasklist /fi "imagename eq notepad.exe"
tasklist /fi "windowtitle eq Untitled*" /v
taskkill /fi "imagename eq notepad.exe"
tasklist /fi "imagename eq notepad.exe"
```

Expected result. Notepad appears in the first list, disappears after `taskkill`. **Explanation.** Without `/f`, Windows asks the window to close; unsaved text may prompt. **Verification.** The last `tasklist` reports no matching tasks. **Troubleshooting.** Windows 11 Notepad may use a different process name pattern; adjust the filter. **Cleanup.** None. **Challenge.** Kill by PID only. **Solution.** Read the PID from `tasklist /fo csv /nh` then `taskkill /pid <n>`.

Lab 83. Detect new processes by diffing snapshots

Objective. Compare two snapshots to find what started. **Difficulty.** [I]. **Prerequisites.** Lab 81. **Environment.** **VM**.

Commands.

```
tasklist /fo csv /nh > snap1.txt
start "" notepad.exe
timeout /t 3 /nobreak >nul
tasklist /fo csv /nh > snap2.txt
fc snap1.txt snap2.txt
findstr /l /x /v /g:snap1.txt snap2.txt
```

Expected result. `fc` shows the `notepad.exe` line (PIDs and memory columns also change); the `findstr /l /x /v /g` command lists lines in snap2 that do not appear verbatim in snap1. **Explanation.** Raw diffs are noisy because memory values change; extract names and PIDs only for a clean diff. **Verification.** Notepad appears. **Troubleshooting.** Close Notepad afterwards. **Cleanup.** `del snap1.txt snap2.txt & taskkill /im notepad.exe`. **Challenge.** Diff only names. **Solution.** `for /f "tokens=1 delims=," %i in (snap1.txt) do @echo %i>> n1.txt` and the same for snap2, then `fc` them.

Lab 84. Service-hosting map with tasklist /svc

Objective. Learn which services share an svchost. **Difficulty.** [I]. **Prerequisites.** Lab 81. **Environment.** Any prompt.

Commands.

```
tasklist /svc /fi "imagename eq svchost.exe"
sc queryex type= service state= running | findstr /i "SERVICE_NAME PID"
```

Expected result. Many svchost lines, each with a PID and a list of service names. **Explanation.** Windows packs related services into shared `svchost.exe` processes. **Verification.** Pick one PID and confirm its services. **Troubleshooting.** The `type= service` form requires the space after `=`. **Cleanup.** None. **Challenge.** Find which process hosts the `Dnscache` service. **Solution.** `tasklist /svc | findstr /i Dnscache`.

Lab 85. Verify an image path and hash

Objective. Check a binary against its expected location and signature. **Difficulty.** [I]. **Prerequisites.** Section 26.2. **Environment.** Any prompt.

Commands.

```
where notepad
certutil -hashfile "%WINDIR%\System32\notepad.exe" SHA256
powershell -NoProfile -Command "(Get-AuthenticodeSignature '%WINDIR%\System32\notepad.exe').Status"
```

Expected result. Path in `System32`, a 64-hex-character hash, and `Valid`. **Explanation.** Path, hash and signature are three independent checks. **Verification.** Repeat for `cmd.exe`. **Troubleshooting.** On Windows 11

the Store version of Notepad lives under `C:\Program Files\WindowsApps`; `where` may show both. **Cleanup.** None. **Challenge.** Explain why a valid signature alone is not proof of safety. **Solution.** Legitimate signed tools can be misused, and stolen certificates exist.

Part IX - Windows Services

The Service Control Manager, service states and accounts, configuration, permissions and safe labs.

Chapter 27. Services Explained

27.1 What a service is

A **service** is a long-running background program managed by the **Service Control Manager (SCM)**, `services.exe`. Services can start at boot with no one logged in, run in **session 0** (no desktop), and run under special accounts. Configuration lives in the registry at `HKLM\SYSTEM\CurrentControlSet\Services\<name>`.

Property	Meaning
Service name	Short internal name (`Spooler`) used by `sc`
Display name	Friendly label shown in Services
Image path (BINARY_PATH_NAME)	The executable and arguments; **must be quoted if it contains spaces**
Start type	`boot`, `system` (drivers), `auto`, `auto (delayed)`, `demand` (manual), `disabled`
Account (SERVICE_START_NAME)	`LocalSystem`, `NT AUTHORITY\LocalService`, `NT AUTHORITY\NetworkService`, a managed service account, or a user
Type	`own process`, `share process` (svchost), `kernel driver`, `file system driver`
Dependencies	Services that must be running first
Recovery actions	What to do on failure (restart, run a program, reboot)
State	`STOPPED`, `START_PENDING`, `RUNNING`, `STOP_PENDING`, `PAUSED`
Security descriptor	Who may start, stop, query or reconfigure it

Service accounts, least to most privileged: **LocalService** (minimal local rights, anonymous on network), **NetworkService** (uses the computer's identity on the network), **LocalSystem** (SYSTEM: full local control). Prefer the least privilege that works; many services should not run as LocalSystem.

A service executable is a special program that **talks to the SCM**; arbitrary programs (like `cmd.exe` or `notepad.exe`) do not respond to the start handshake, so the SCM fails them with error 1053 (the service did not respond in time).

27.2 Why services matter to security

Services run with high privilege, start before logon and survive reboots, so they are both the engine of the OS and a classic place to look for unauthorized persistence or privilege escalation. The defensive checklist:

1. **Unquoted service paths** with spaces (`C:\Program Files\My App\svc.exe` unquoted) create ambiguous resolution; Windows may try `C:\Program.exe` first.
2. **Writable binaries or folders**: if a low-privilege user can replace the service executable or a folder on its path, they can run code as the service account.
3. **Weak service permissions**: if ordinary users may reconfigure a service (`SERVICE_CHANGE_CONFIG`), they can point it at another binary.
4. **New services** installed unexpectedly (event 7045 in the System log; 4697 in Security when auditing is on).

5. **Services whose binary is missing or in user-writable places** (`%TEMP%`, `AppData`, `ProgramData` with weak ACLs, `C:\Users\Public`).

Part XIX shows how to detect each condition safely.

SC [External, Service Control]

Field	Detail
Purpose / Why	Query, start, stop, configure, create and delete services; read and set service security descriptors
Syntax	`sc [\\server] <command> [service] [options]` ; commands `query`, `queryex`, `qc`, `qdescription`, `start`, `stop`, `pause`, `continue`, `config`, `create`, `delete`, `failure`, `sdshow`, `sdset`
Key switches	`sc query state= all type= service` (**note the space after each** =), `sc qc name`, `sc queryex name` (adds PID), `sc config name start= demand`
Beginner	`sc query Spooler`
Intermediate	`sc qc Spooler`
Advanced	`sc sdshow Spooler` (security descriptor in SDDL) ; `sc queryex type= service state= running`
Real-world	Checking dependencies before a patch; confirming a service's account and binary during an audit
Output	`SERVICE_NAME`, `TYPE`, `STATE` (and `WIN32_EXIT_CODE`, `PID` with `queryex`). `qc` shows `START_TYPE`, `BINARY_PATH_NAME`, `SERVICE_START_NAME`, `DEPENDENCIES`.
Exit / Errors	5 access denied; 1060 service does not exist; 1053 service did not respond to the start request; 1056 already running; 1058 disabled
Admin	Query: no for most services. Start/stop/config/create/delete: yes (unless the ACL grants you the right)
Versions	Same; **in PowerShell** `sc` **is an alias for** `Set-Content`**; type** `sc.exe`
Mistakes	Forgetting the space after `=`; changing start type of core services; stopping a service others depend on
Security	Compare `BINARY_PATH_NAME` with disk reality; `sc sdshow` reveals who may control the service; new or changed services are a prime persistence indicator
Lab / Cleanup	Labs 86-88
Related	`net start/stop`, `tasklist /svc`, `reg query HKLM\SYSTEM\CurrentControlSet\Services`
PowerShell	`Get-Service`, `Set-Service`, `New-Service`, `Get-CimInstance Win32_Service`

NET START, NET STOP, NET PAUSE, NET CONTINUE [External, services (legacy)]

Field	Detail
Purpose / Why	Older, simpler service control by **display name** or service name
Syntax	`net start [service]`, `net stop service`, `net pause service`, `net continue service`
Beginner	`net start` (lists running services by display name)

Field	Detail
Intermediate	`net stop "Print Spooler"` then `net start "Print Spooler"`
Advanced	`net stop Spooler /y` stops dependents too
Real-world	Quick restart in a script
Output	`The Print Spooler service was stopped successfully.`
Exit / Errors	2 (various, see `net helpmsg N`); `The service name is invalid.`; `System error 5 has occurred. Access is denied.`
Admin	Yes to start/stop
Versions	Works on 10 and 11; `sc` gives more detail
Mistakes	Using display names that differ per language
Security	`net helpmsg 2182` decodes a message number; `net` commands are heavily logged by security tools
Lab / Cleanup	Lab 86 uses read-only plus one benign stop/start of Print Spooler on a VM
Related	`sc`, `Restart-Service`
PowerShell	`Start-Service`, `Stop-Service`, `Restart-Service`

Chapter 28. Service Labs

> **SAFETY.** Service labs use read-only queries and one disposable service entry that is **never started**. Run create/delete steps only in a VM with a snapshot.

Lab 86. Read a service completely

Objective. Understand every field of a service. **Difficulty.** [B]. **Prerequisites.** Chapter 27. **Environment.** Any prompt (elevated only for the optional restart).

Commands.

```
sc query Spooler
sc queryex Spooler
sc qc Spooler
sc qdescription Spooler
sc query state= all type= service | find /c "SERVICE_NAME"
```

Expected result. Service state (`RUNNING` or `STOPPED`), PID, type `WIN32_OWN_PROCESS`, start type `AUTO_START`, binary `C:\WINDOWS\System32\spoolsv.exe`, account `LocalSystem`, and a short dependency list (for example `RPCSS`; exact entries vary by build). The final count shows how many services exist. **Explanation.** `qc` is configuration; `query` is live state. **Verification.** Compare with the GUI (`services.msc`). **Troubleshooting.** `OpenService FAILED 1060` means a mistyped name; use `sc query state= all` to find the short name. **Cleanup.** None. **Challenge.** List only auto-start services. **Solution.** `sc query state= all type= service` does not show start type; use `powershell -NoProfile -Command "Get-CimInstance Win32_Service | ? StartMode -eq Auto | select Name"`.

Lab 87. Create and delete a benign service entry (never start it)

Objective. See what `sc create` writes and how to remove it. **Difficulty.** [I]. **Prerequisites.** Lab 86. **Environment. VM + snapshot**, elevated.

Commands.

```
sc create CmdLabSvc binPath= "%WINDIR%\System32\cmd.exe" start= demand DisplayName= "CMD Lab Service"
sc qc CmdLabSvc
sc query CmdLabSvc
reg query HKLM\SYSTEM\CurrentControlSet\Services\CmdLabSvc
sc delete CmdLabSvc
sc query CmdLabSvc
```

Expected result. `[SC] CreateService SUCCESS`; `qc` shows start type `DEMAND_START`, state `STOPPED`; the registry key lists `ImagePath`, `Start`, `ObjectName`; after delete the final query returns error 1060. **Explanation.** A service is a registry entry; `cmd.exe` would not answer the SCM handshake, so **do not start it**. **Verification.** Check the System log for event 7045 (service installed): `wevtutil qe System "/q:*[System[(EventID=7045)]]" /c:3 /rd:true /f:text`. **Troubleshooting.** If delete reports `marked for deletion`, close any window showing services and reboot. **Cleanup.** The delete step. **Challenge.** Find the creation event. **Solution.** Event ID 7045 in the System log shows the service name, image path and account.

Lab 88. Audit services for risky paths (read-only)

Objective. Find unquoted paths and binaries in user-writable folders. **Difficulty.** [A]. **Prerequisites.** Section 27.2. **Environment.** Any prompt; read-only.

Commands.

```
powershell -NoProfile -Command "Get-CimInstance Win32_Service | Where-Object { $_.PathName -and $_.PathName -notm
powershell -NoProfile -Command "Get-CimInstance Win32_Service | Where-Object { $_.PathName -match 'Users|Temp|App
```

Expected result. Ideally nothing. Any entry needs follow-up. **Explanation.** Unquoted paths with spaces and services living in user-writable places are standard review findings. **Verification.** For each finding run `sc qc <name>` and `icacls` on the folder. **Troubleshooting.** Some vendor services legitimately have unquoted paths that do not contain spaces before the executable (the filter requires a space and no leading quote. The `\x22` in the regex stands for a double-quote character; it avoids nesting quotes inside the CMD command line). **Cleanup.** None. **Challenge.** Save the findings to CSV. **Solution.** Append `| Export-Csv -NoTypeInformation findings.csv`.

Lab 89. Dependencies and safe stop/start

Objective. Predict the impact of stopping a service. **Difficulty.** [I]. **Prerequisites.** Lab 86. **Environment. VM**, elevated.

Commands.

```
sc enumdepend Spooler
net stop Spooler
sc query Spooler
net start Spooler
sc query Spooler
```

Expected result. Dependents (if any) listed; stop succeeds; state `STOPPED`, then `RUNNING`. **Explanation.** `enumdepend` shows who relies on this service; stopping it will stop them too. **Verification.** The two `query` outputs. **Troubleshooting.** If printing is in use, stop may take time. **Cleanup.** The restart is the cleanup. **Challenge.** Add a retry if the service is `STOP_PENDING`. **Solution.** Loop with `sc query Spooler | find "STOPPED"` and `ping -n 2 127.0.0.1 >nul`, up to a counter.

Lab 90. Service change capture (baseline diff)

Objective. Detect a new or changed service by diffing exports. **Difficulty.** [I]. **Prerequisites.** Lab 87. **Environment. VM**, elevated.

Commands.

```
sc query state= all type= service > svc1.txt
sc create CmdLabSvc binPath= "%WINDIR%\System32\cmd.exe" start= demand >nul
sc query state= all type= service > svc2.txt
fc svc1.txt svc2.txt
sc delete CmdLabSvc
```

Expected result. `fc` highlights the added `CmdLabSvc` entry. **Explanation.** Even a crude diff finds additions between known-good and current state. **Verification.** Only that service differs (other services may change state). **Troubleshooting.** Pipe both files through `find "SERVICE_NAME"` before comparing to reduce noise. **Cleanup.** `del svc1.txt svc2.txt`. **Challenge.** Do the same for the registry hive of services. **Solution.** `reg export HKLM\SYSTEM\CurrentControlSet\Services s1.reg` before and after, then `fc`.

Part X - Users, Groups and Security Tokens

Who you are to Windows: accounts, SIDs, groups, privileges, integrity levels and UAC, with line-by-line reading of `whoami /all`.

Chapter 29. Identity, Tokens and UAC

29.1 Accounts and groups

- A **local account** lives in the machine's SAM database; a **domain account** lives in Active Directory; a **Microsoft account** is linked to a local profile.
- A **group** is a set of SIDs used to grant access in bulk. Important local groups: `Administrators`, `Users`, `Remote Desktop Users`, `Backup Operators`, `Guests`.
- Built-in accounts: `Administrator` (often disabled), `Guest` (disabled), `DefaultAccount`, `WDAGUtilityAccount`; **service identities** `SYSTEM`, `LOCAL SERVICE`, `NETWORK SERVICE`.
- **SID format:** `S-1-5-21-<domain-or-machine id>-<RID>`. The last number is the **RID**: 500 is the built-in Administrator, 501 Guest, 1000+ ordinary accounts. Renaming the Administrator account does not change its SID.

29.2 The access token

At logon Windows builds an **access token** and attaches it to every process the user starts. It contains:

Token field	Meaning
User SID	Who you are
Group SIDs (with attributes)	Every group you belong to; each may be enabled, **deny-only**, mandatory or owner
Privileges	System-wide rights such as `SeShutdownPrivilege`, `SeDebugPrivilege`; **enabled or disabled**
Integrity level	Mandatory label SID (Low, Medium, High, System)
Logon session ID	Ties token to a logon
Token type	Primary (for processes) or impersonation (for threads acting for another identity)
Restricting SIDs	Present in **restricted tokens** (used by sandboxes)

Privileges are not permissions. A privilege is a right to do a system-wide operation (back up files ignoring ACLs, debug other processes, load drivers). A permission is an ACL entry on a specific object. Having `SeDebugPrivilege` listed and **disabled** is normal for an elevated administrator token until a program enables it; the dangerous state is a program that does enable it.

29.3 UAC and the two tokens

When an administrator logs on, Windows creates a **full token** and a **filtered token** (administrator groups reduced to deny-only; most privileges removed; Medium integrity). Normal programs get the filtered token. When you approve a UAC prompt ("Run as administrator") the elevated program gets the **full token** at **High** integrity.

	Filtered (standard)	Elevated
Integrity	Medium (8192)	High (12288)
`Administrators` group	`Group used for deny only`	`Enabled`

	Filtered (standard)	Elevated
Privileges	few	many (mostly shown Disabled until used)
File/registry virtualization for legacy apps	possible	no

Integrity levels: Untrusted 0, **Low 4096** (S-1-16-4096), **Medium 8192**, **High 12288**, **System 16384**. A process at a lower level cannot write to objects at a higher level (mandatory integrity control), regardless of ACL.

UAC is not a security boundary against a determined attacker already running as the user; Microsoft states this explicitly in its UAC documentation and servicing criteria. It is a defense that makes silent elevation less likely and gives consent prompts. The strong boundaries are separate accounts, separate machines and virtualization.

WHOAMI [External, identity]

Field	Detail
Purpose / Why	Shows the current user, groups, privileges, logon ID and integrity level
Syntax	`whoami [/user] [/groups] [/priv] [/logonid] [/all] [/fo table|list|csv] [/upn] [/fqdn]`
Beginner	`whoami`
Intermediate	`whoami /groups`
Advanced	`whoami /all /fo list`
Real-world	First check when "access denied" appears: is the token elevated, and is the needed group present?
Output	See Section 29.4
Exit / Errors	0; `ERROR: Unable to get ...` on locked-down systems
Admin	No
Versions	Present since Vista
Mistakes	Confusing "member of Administrators" with "running elevated"; reading `whoami /priv` as a list of what the program is doing
Security	Group membership changes alter token content only at **next logon**; a stale token explains "I added myself but it still fails"
Lab / Cleanup	Lab 91
Related	`net user`, `net localgroup`, `runas`, `icacls`
PowerShell	`[Security.Principal.WindowsIdentity]::GetCurrent()`, `Get-LocalUser`

29.4 Reading `whoami /all` line by line

```
USER INFORMATION
User Name        SID
================ ============================================
desktop-01\alice S-1-5-21-1111111111-2222222222-3333333333-1001
```

The account name and its SID (RID 1001 is the first ordinary local account).

```
GROUP INFORMATION
Group Name                             Type             SID          Attributes
====================================== ================ ============ ==================================
Everyone                               Well-known group S-1-1-0      Mandatory group, Enabled by default, Enabled g
BUILTIN\Administrators                 Alias            S-1-5-32-544 Group used for deny only
BUILTIN\Users                          Alias            S-1-5-32-545 Mandatory group, Enabled by default, Enabled g
NT AUTHORITY\INTERACTIVE               Well-known group S-1-5-4      Mandatory group, Enabled by default, Enabled g
NT AUTHORITY\Authenticated Users       Well-known group S-1-5-11     Mandatory group, Enabled by default, Enabled g
Mandatory Label\Medium Mandatory Level Label            S-1-16-8192  Mandatory group, Enabled by default, Enabled g
```

Key reading: `Group used for deny only` on Administrators means this is the **filtered** token of an admin; in an elevated prompt the same line says `Enabled group`. `Mandatory Label\...` is the integrity level.

```
PRIVILEGES INFORMATION
Privilege Name                Description                          State
============================= ==================================== ========
SeShutdownPrivilege           Shut down the system                 Disabled
SeChangeNotifyPrivilege       Bypass traverse checking             Enabled
SeUndockPrivilege             Remove computer from docking station Disabled
SeIncreaseWorkingSetPrivilege Increase a process working set       Disabled
SeTimeZonePrivilege           Change the time zone                 Disabled
```

A standard token has few privileges. An elevated one adds entries such as `SeDebugPrivilege`, `SeBackupPrivilege`, `SeRestorePrivilege`, `SeTakeOwnershipPrivilege`, `SeLoadDriverPrivilege`, mostly **Disabled** until a program enables them. **State is not the same as capability.**

NET USER and NET LOCALGROUP [External, accounts]

Field	Detail
Purpose / Why	List, create, modify and delete local users and local group membership
Syntax	`net user [name [password|*] [/add] [/delete] [/active:yes|no]]` ; `net localgroup [group [name /add|/delete]]`
Beginner	`net user` ; `net localgroup`
Intermediate	`net user alice` (detail: last logon, groups, password dates)
Advanced	`net localgroup Administrators` (audit who is an admin)
Real-world	Weekly review of the local Administrators group on a standalone server
Output	Account name, full name, active state, password last set/expires, last logon, local and global group memberships
Exit / Errors	2 and `System error 5 has occurred. Access is denied.` without elevation; `System error 1378` already a member; `System error 1387` no such member
Admin	Yes to change; reading is allowed
Versions	Same; domain variants use `/domain`
Mistakes	**Putting a plaintext password on the command line** (visible in process listings and command-line logs); use `*` so CMD prompts. Deleting the wrong account removes its SID and profile ownership.
Security	Membership of Administrators is a top audit item; events 4720 (user created), 4722 (enabled), 4724 (password reset), 4726 (deleted), 4732 (added to local group), 4728/4756 (domain groups). Not shown on `net user`: the `Authenticated Users`/`Everyone` pseudo-groups
Lab / Cleanup	Lab 92

Field	Detail
Related	`whoami`, `wmic useraccount` (deprecated), `lusrmgr.msc`
PowerShell	`Get-LocalUser`, `New-LocalUser`, `Get-LocalGroupMember`, `Add-LocalGroupMember`

RUNAS [External, alternate credentials]

Field	Detail
Purpose / Why	Starts a program under a different account (prompting for that account's password)
Syntax	`runas /user:name [/netonly] [/noprofile] "command"`
Key switches	`/netonly` use the credentials only for network access; `/noprofile` skip loading the profile
Beginner	`runas /user:LabUser cmd`
Intermediate	`runas /user:%COMPUTERNAME%\LabUser "cmd /k whoami"`
Advanced	`runas /netonly /user:DOMAIN\admin "mmc.exe"`
Real-world	Testing what a standard account can and cannot do
Output	`Enter the password for LabUser:` then a new window
Exit / Errors	`RUNAS ERROR: Unable to run - 1326: The user name or password is incorrect.`; fails if the Secondary Logon service is disabled
Admin	No (needs the other account's credentials)
Versions	Same
Mistakes	`/savecred` **stores the password in the credential manager so anyone using your session can reuse it silently; do not use it.** `runas` does not elevate; it logs on as another user.
Security	Run test tools as a standard account and avoid daily use of an admin account; logons appear as event 4648 (explicit credentials) and 4624
Lab / Cleanup	Lab 92
Related	`whoami`, Run as administrator (UAC)
PowerShell	`Start-Process -Credential`, `Start-Process -Verb RunAs` (UAC elevation)

Lab 91. Read your token

Objective. Interpret `whoami /all` in a standard and an elevated prompt. **Difficulty.** [I]. **Prerequisites.** Section 29.4. **Environment. VM**, two prompts.

Commands. In each prompt:

```
whoami /user
whoami /groups | findstr /i "Administrators Mandatory"
whoami /priv
```

Expected result. Standard: Administrators `Group used for deny only`, Medium label, few privileges. Elevated: `Enabled group`, High label, many privileges (mostly Disabled). **Explanation.** Same user, different tokens. **Verification.** Count privileges in each (`whoami /priv | find /c "Se"`). **Troubleshooting.** A

non-admin account will never show the elevated variant. **Cleanup.** Close the elevated prompt. **Challenge.** Identify the logon session ID. **Solution.** `whoami /logonid`.

Lab 92. Create a test user, use runas, then remove it

Objective. Create a throwaway standard account and see its token. **Difficulty.** [I]. **Prerequisites.** Lab 91. **Environment. VM + snapshot**, elevated.

Commands.

```
net user LabUser * /add
net user LabUser
net localgroup Users
runas /user:%COMPUTERNAME%\LabUser "cmd /k whoami /groups"
net user LabUser /delete
net user LabUser
```

Expected result. You are prompted twice for the password (type a throwaway value; it is not echoed); `net user LabUser` shows the account in `Users`; the new window lists groups with a Medium label and no Administrators entry; after delete, `The user name could not be found.` **Explanation.** A standard account gets a token without admin groups. **Verification.** Events 4720 and 4726 in the Security log if auditing is on. **Troubleshooting.** `runas` fails if the Secondary Logon service is stopped (`net start seclogon`). **Cleanup.** The delete step; also remove the profile folder `C:\Users\LabUser` if it was created. **Challenge.** Add LabUser to Administrators and watch the change in `whoami /groups` of a new `runas` session. **Solution.** `net localgroup Administrators LabUser /add`; the new session shows `Group used for deny only` unless elevated; remove with `/delete`.

Lab 93. Audit local administrators

Objective. List who can administer this machine. **Difficulty.** [B]. **Prerequisites.** `net localgroup` card. **Environment.** Any prompt.

Commands.

```
net localgroup Administrators
net user Administrator | findstr /i "active"
net user Guest | findstr /i "active"
```

Expected result. A short membership list; built-in Administrator and Guest typically `Account active No`. **Explanation.** Unexpected members or active default accounts are findings. **Verification.** Compare against your inventory. **Troubleshooting.** Names are localized on non-English systems. **Cleanup.** None. **Challenge.** Export to a dated file. **Solution.** `net localgroup Administrators > admins_%DATE:~-4%%DATE:~4,2%%DATE:~7,2%.txt` (US date format assumed; see Section 22.7 for a locale-proof stamp).

Lab 94. Integrity levels in action

Objective. Show that Medium integrity cannot write to High-integrity locations even if ACLs are loosened. **Difficulty.** [A]. **Prerequisites.** Section 29.3. **Environment. VM**, standard prompt.

Commands.

```
whoami /groups | find "Mandatory Label"
icacls "%WINDIR%\System32" | findstr /i "Users"
echo test> "%WINDIR%\System32\labfile.txt"
echo %ERRORLEVEL%
```

Expected result. Medium label; `Users` only has `(RX)`; the write fails with `Access is denied.` and a nonzero code. **Explanation.** Standard users lack write permission, and the integrity system adds a second wall for lower-integrity processes. **Verification.** In an elevated prompt the same command succeeds; delete the file. **Troubleshooting.** If it succeeds in a standard prompt, you are logged on as the built-in Administrator or UAC is off. **Cleanup.** `del %WINDIR%\System32\labfile.txt` (elevated). **Challenge.** Explain why UAC-off is dangerous. **Solution.** All programs run with the full admin token, so any malicious program has full power and there are no prompts.

Lab 95. Privilege reading drill

Objective. Decide whether a privilege list indicates trouble. **Difficulty.** [A]. **Prerequisites.** Lab 91. **Environment. VM**, elevated.

Commands.

```
whoami /priv /fo csv /nh > privs.csv
findstr /i "SeDebug SeBackup SeRestore SeTakeOwnership SeLoadDriver SeImpersonate" privs.csv
```

Expected result. Each of those privileges present in an elevated admin token with state `Disabled`. **Explanation.** Their presence is expected for administrators; the investigation signal is a process **using** them (for example `SeDebugPrivilege` enabled in an unexpected process), which requires process telemetry (Sysmon, EDR, event 4703 token-right adjustment when audited). **Verification.** The same command in a standard prompt finds far fewer. **Troubleshooting.** CSV quoting is normal. **Cleanup.** `del privs.csv`. **Challenge.** Write a three-sentence note explaining the difference between privilege and permission. **Solution.** A permission is an ACL entry on a specific object; a privilege is a system-wide right in the token; privileges can override some ACL checks (backup/restore), so they are tightly controlled.

Part XI - The Windows Registry

The hierarchical configuration database: hives, keys, values, permissions, virtualization, backup and restore, and the places defenders look.

Chapter 30. Registry Architecture

30.1 What the registry is

The **registry** is a hierarchical database that stores configuration for Windows, services, drivers, applications and users. It is organized like a file system: **keys** are like folders, **values** are like files inside them, each value has a **name**, a **type** and **data**. Every key has an ACL, just like a file.

30.2 The five root keys and the real hives

Only two roots are real storage; the others are views.

Root key	Abbrev.	What it really is
`HKEY_LOCAL_MACHINE`	HKLM	Machine-wide settings. Backed by hive files.
`HKEY_USERS`	HKU	One subkey per loaded user profile, named by SID, plus `.DEFAULT`
`HKEY_CURRENT_USER`	HKCU	An alias for `HKU\<your SID>` (backed by `NTUSER.DAT` in your profile)
`HKEY_CLASSES_ROOT`	HKCR	A **merged view** of `HKLM\SOFTWARE\Classes` and `HKCU\Software\Classes` (user wins)
`HKEY_CURRENT_CONFIG`	HKCC	A view of the current hardware profile under HKLM

Main hive files (under `%SystemRoot%\System32\config` unless noted):

Hive (mounted at)	File
`HKLM\SYSTEM`	`SYSTEM` (services, drivers, control sets)
`HKLM\SOFTWARE`	`SOFTWARE` (installed software, Windows settings)
`HKLM\SAM`	`SAM` (local account database; restricted)
`HKLM\SECURITY`	`SECURITY` (local security policy; restricted)
`HKU\.DEFAULT`	`DEFAULT`
`HKU\<SID>` / `HKCU`	`%USERPROFILE%\NTUSER.DAT`
`HKCU\Software\Classes`	`%LOCALAPPDATA%\Microsoft\Windows\UsrClass.dat`
`HKLM\HARDWARE`	volatile, built at boot (no file)

30.3 Value types

Type	Meaning	Example
`REG_SZ`	Text string	`Hello`
`REG_EXPAND_SZ`	Text with `%VAR%` references expanded by the consumer	`%SystemRoot%\system32`
`REG_MULTI_SZ`	List of strings	dependency lists
`REG_DWORD`	32-bit number	flags, `Start` values
`REG_QWORD`	64-bit number	timestamps, sizes

Type	Meaning	Example
`REG_BINARY`	Raw bytes	hardware data
`REG_NONE` / link types	Special	rare

Every key also has an unnamed **default value** (shown as `(Default)`; `reg` calls it `/ve`).

30.4 32-bit and 64-bit views, and virtualization

On 64-bit Windows, 32-bit programs see a redirected view: their `HKLM\SOFTWARE` is really `HKLM\SOFTWARE\WOW6432Node`. Use `reg query ... /reg:32` or `/reg:64` to choose the view explicitly.

Separately, **UAC virtualization** (legacy, for 32-bit apps that are not UAC-aware) silently redirects writes to protected locations to per-user areas: `%LOCALAPPDATA%\VirtualStore` for files and `HKCU\Software\Classes\VirtualStore` for registry. If an old program "saves" settings that other tools cannot see, check there. Virtualization applies only to non-elevated legacy processes.

30.5 Permissions and risk

Registry keys have DACLs and owners, so the principles of Chapter 23 apply. **Writable registry keys that affect privileged code are a vulnerability class** (for example, a service's `ImagePath` key writable by Users). Read key ACLs with `reg query` plus PowerShell `Get-Acl`, or Sysinternals `AccessChk`; there is no `icacls` for the registry.

30.6 Backup and restore options

Method	Command	Notes
Text export (`.reg`)	`reg export KEY file.reg /y`	Human-readable; import with `reg import`; Unicode text
Binary hive save	`reg save KEY file.hiv /y`	Exact copy of a key tree as a hive; needs SeBackupPrivilege for some hives
Binary restore	`reg restore KEY file.hiv`	**Replaces** the key's content; needs SeRestorePrivilege; risky
System Restore, VSS, Windows Backup	various	Whole-system approach

Always export before editing. Test on disposable keys. Do not edit the registry of a production system without a rollback plan.

> **DANGER / CAUTION.** Never run `reg delete` on a root or high-level key (`HKLM\SOFTWARE`, `HKLM\SYSTEM`, `HKCU\Software`) and never import a `.reg` file you have not read. A `.reg` file can add, change or delete anywhere you have permission.

30.7 Security-relevant locations (audit checklist)

These keys control what runs or how components are resolved. **Defenders review them; this is not a how-to for modifying them.**

Area	Typical location	What to look for
Autostart (machine)	`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`, `RunOnce`, `...\Policies\Explorer\Run`	Entries pointing to user-writable paths or unknown binaries
Autostart (user)	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, `RunOnce`	Same; user-level needs no admin, so it is a common abuse spot
32-bit autostart	`HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run`	Same
Services and drivers	`HKLM\SYSTEM\CurrentControlSet\Services\<name>`	`ImagePath`, `Start`, `ObjectName`; unexpected new keys
Winlogon	`HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon`	Altered `Shell` / `Userinit` values
Image File Execution Options	`HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\<exe>`	Unexpected `Debugger` values
File associations	`HKLM\SOFTWARE\Classes`, `HKCU\Software\Classes` (`<type>\shell\open\command`)	Handlers launching unexpected programs
AppInit / LSA / providers	`HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows`, `HKLM\SYSTEM\CurrentControlSet\Control\Lsa`	Added DLLs or packages
Scheduled tasks cache	`HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache`	Tasks present in registry but missing in the UI
Policy locations	`HKLM\SOFTWARE\Policies`, `HKCU\Software\Policies`	Settings that disable security features (check Defender, Firewall, UAC keys)

Typical review technique: **export, diff against a baseline, investigate differences, verify each referenced file exists, is signed and lives in a protected folder.** Tools such as Sysinternals Autoruns automate this; the CMD approach teaches what Autoruns is reading.

REG [External, registry]

Field	Detail
Purpose / Why	Read, create, change, delete, export, import, compare, copy, save, restore, load and unload registry data from the command line
Syntax	`reg query KEY [/v name | /ve] [/s] [/f text [/k] [/d] [/c] [/e]] [/t type] [/reg:32|/reg:64]`; `reg add KEY [/v name | /ve] [/t type] [/d data] [/f]`; `reg delete KEY [/v name | /ve | /va] [/f]`; `reg export KEY file [/y]`; `reg import file`; `reg copy KEY1 KEY2 [/s] [/f]`; `reg compare KEY1 KEY2 [/s]`; `reg save KEY file [/y]`; `reg restore KEY file`; `reg load KEY file`; `reg unload KEY`
Key switches	`/s` recurse, `/f` (query: search text; add/delete/copy: no confirmation), `/v` value name, `/ve` default value, `/t` type, `/d` data, `/k` search key names, `/d` (in query with `/f`) search data
Beginner	`reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion" /v ProductName`

Field	Detail
Intermediate	`reg add HKCU\Software\CmdLab /v Mode /t REG_SZ /d test /f` then `reg query HKCU\Software\CmdLab`
Advanced	`reg query HKLM\SYSTEM\CurrentControlSet\Services /s /f ImagePath /v` (find every service image path)
Real-world	Checking an application's configuration value across many machines; capturing a key before changing it
Output	Key name, then value lines: `Name Type Data`. Exports are Unicode `.reg` text.
Exit / Errors	0 success; 1 failure (message like `ERROR: The system was unable to find the specified registry key or value.` or `ERROR: Access is denied.`); `compare`: 0 identical, 2 differences
Admin	Not for HKCU; yes for most of HKLM (and `save`/`restore`/`load`/`unload`)
Versions	Same on 10/11; `reg` is not subject to WMIC deprecation
Mistakes	`reg add` without `/f` prompts in scripts; `reg delete KEY /va` deletes **all** values; %VAR% in `/d` for REG_EXPAND_SZ is expanded by **CMD first** (write ^%VAR^% at the prompt or %%VAR%% in a script); forgetting the WOW64 view; line length limits for long data
Security	Registry commands are common in telemetry; changes under `Run`, `Services`, `Policies`, `Image File Execution Options` should be investigated. Never store passwords in the registry via scripts.
Lab / Cleanup	Labs 96-98; always delete `HKCU\Software\CmdLab` afterward
Related	`setx`, `regedit` (GUI), `schtasks`, `sc`
PowerShell	`Get-ItemProperty`, `Set-ItemProperty`, `New-ItemProperty`, `Remove-Item`; the registry appears as drives `HKLM:` and `HKCU:`

SETX [External, persistent environment variables]

Field	Detail
Purpose / Why	Writes a variable to the registry so **future** processes inherit it (user: `HKCU\Environment`; machine: `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment`)
Syntax	`setx name value [/m]`
Beginner	`setx LABVAR hello`
Intermediate	`setx LABVAR "two words"`
Advanced	`setx LABVAR ""` clears the value to empty (does not delete the entry; delete it with `reg delete HKCU\Environment /v LABVAR /f`)
Real-world	Setting a tool's home directory for all new sessions
Output	`SUCCESS: Specified value was saved.`
Exit / Errors	0 or 1
Admin	`/m` (machine) needs elevation
Versions	Same
Mistakes	**The current window is not updated**; open a new one. `setx PATH "%PATH%"` flattens user and machine PATH together and can truncate at 1024 characters on older builds; avoid it.
Security	Persistent variables such as `PATH` ordering decide which tool runs; machine-wide changes need review

Field	Detail
Lab / Cleanup	Lab 96; delete the variable afterward
Related	`set`, `reg`, `where`
PowerShell	`[Environment]::SetEnvironmentVariable(name, value, 'User')`

Lab 96. Read the registry without changing it

Objective. Query keys, values and a search. **Difficulty.** [B]. **Prerequisites.** Chapter 30. **Environment.** Standard prompt.

Commands.

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion" /v ProductName
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion" /v CurrentBuildNumber
reg query HKCU\Environment
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
reg query HKLM\SYSTEM\CurrentControlSet\Services\Spooler /v ImagePath
reg query HKLM\SYSTEM\CurrentControlSet\Services\Spooler /v Start
```

Expected result. Product name and build; your user variables; your own autostart entries (possibly none: `ERROR: The system was unable to find the specified registry key or value.` is a normal "none"); Spooler `ImagePath` and `Start 0x2` (automatic). **Explanation.** Read-only queries are safe anywhere. `Start` values: 0 boot, 1 system, 2 auto, 3 manual, 4 disabled. **Verification.** Compare with `sc qc Spooler`. **Troubleshooting.** Quote any key that has spaces. **Cleanup.** None. **Challenge.** List all service image paths containing `Program Files`. **Solution.** `reg query HKLM\SYSTEM\CurrentControlSet\Services /s /v ImagePath | findstr /i "Program Files"`.

Lab 97. Disposable key round trip: add, export, change, import, compare, delete

Objective. Practice every editing verb on a throwaway key. **Difficulty.** [I]. **Prerequisites.** Lab 96. **Environment.** Standard prompt, scratch folder.

Commands.

```
reg add HKCU\Software\CmdLab /v Name /t REG_SZ /d original /f
reg add HKCU\Software\CmdLab /v Count /t REG_DWORD /d 5 /f
reg add HKCU\Software\CmdLab\Sub /v Note /t REG_SZ /d child /f
reg query HKCU\Software\CmdLab /s
reg export HKCU\Software\CmdLab before.reg /y
reg add HKCU\Software\CmdLab /v Name /t REG_SZ /d modified /f
reg export HKCU\Software\CmdLab after.reg /y
fc /u before.reg after.reg
reg import before.reg
reg query HKCU\Software\CmdLab /v Name
reg copy HKCU\Software\CmdLab HKCU\Software\CmdLab2 /s /f
reg compare HKCU\Software\CmdLab HKCU\Software\CmdLab2 /s
echo compare exit code %ERRORLEVEL%
reg delete HKCU\Software\CmdLab /f
reg delete HKCU\Software\CmdLab2 /f
```

Expected result. The query shows both values and the subkey; `fc` highlights the `Name` change; after import `Name` is `original`; `reg compare` reports `Result Compared: Identical` and the exit code line shows `0`; both keys are deleted at the end. **Explanation.** Export/import is the quick undo for edits. **Verification.** `reg query HKCU\Software\CmdLab` finally reports not found. **Troubleshooting.** `.reg` files are UTF-16 text, which is why the command uses `fc /u` (Unicode comparison). **Cleanup.** `del before.reg after.reg`. **Challenge.** Show that `REG_EXPAND_SZ` is expanded by CMD first. **Solution.** `reg add HKCU\Software\CmdLab /v P /t REG_EXPAND_SZ /d %TEMP% /f` stores the resolved path; with `^%TEMP^%` it stores the literal reference. Delete the key when done.

Lab 98. Save, load and unload a hive (VM, elevated)

Objective. Mount a saved hive under a temporary name and inspect it offline. **Difficulty.** [A]. **Prerequisites.** Lab 97. **Environment. VM + snapshot**, elevated.

Commands.

```
reg add HKCU\Software\CmdLab /v Secret /t REG_SZ /d lab-only-value /f
reg save HKCU\Software\CmdLab "%TEMP%\cmdlab.hiv" /y
reg delete HKCU\Software\CmdLab /f
reg load HKLM\CmdLabHive "%TEMP%\cmdlab.hiv"
reg query HKLM\CmdLabHive
reg unload HKLM\CmdLabHive
del "%TEMP%\cmdlab.hiv"
```

Expected result. The loaded hive shows `Secret`; after `unload` the key is gone. **Explanation.** This is exactly how offline registry files (`SOFTWARE` from a dead system, a user's `NTUSER.DAT`) are examined in forensic and recovery work. **Verification.** `reg query HKLM\CmdLabHive` fails after unload. **Troubleshooting.** `unload` fails with `Access is denied` if a command prompt or tool still holds a handle inside the key: `cd` out of it and close regedit. **Cleanup.** Already done; confirm with `dir "%TEMP%\cmdlab.hiv"`. **Challenge.** Explain why you must unload before using the file. **Solution.** A loaded hive is locked and may be flushed late; copying a loaded hive can give an inconsistent file.

Part XII - Scheduled Tasks

The Task Scheduler: architecture, triggers, actions, principals, history, XML, auditing and safe labs.

Chapter 31. Task Scheduler Architecture and schtasks

31.1 Architecture

The **Task Scheduler** service (`Schedule`, running in a `svchost.exe`) runs tasks when conditions are met. A task consists of:

Part	Meaning
Triggers	When: at a time, daily/weekly/monthly, at logon, at startup, on idle, on an event-log event, on workstation lock/unlock
Actions	What: start a program (the common one); older send-mail and message actions are deprecated
Principal	Who: user account or `SYSTEM`, and run level (`LIMITED` or `HIGHEST`, which means elevated)
Conditions	Power state, idle, network
Settings	Allow on demand, run missed tasks, stop if running too long, restart on failure, **hidden**
History	Optional event recording (off by default)

Storage:

- XML definitions: `%SystemRoot%\System32\Tasks\<folder>\<task>`.
- Registry cache: `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\{Tree, Tasks, Plain}`.
- Evidence of task activity: **Microsoft-Windows-TaskScheduler/Operational** log (events 106 registered, 140 updated, 141 deleted, 200 action started, 201 action completed, 100/102 task started/finished) **only if history is enabled**: `wevtutil sl Microsoft-Windows-TaskScheduler/Operational /e:true` (admin). Security log events 4698 created, 4699 deleted, 4700 enabled, 4701 disabled, 4702 updated, when "Audit Other Object Access Events" is on.

Tasks running as SYSTEM execute with the highest local privileges and survive reboots; that is why they are both essential administration tools and a common abuse target.

SCHTASKS [External, Task Scheduler]

Field	Detail
Purpose / Why	Create, query, run, change, end and delete scheduled tasks
Syntax	`schtasks /query [/tn name] [/fo table|list|csv] [/v] [/xml]`; `schtasks /create /tn name /tr "command" /sc type [/st hh:mm] [/ru user] [/rl highest|limited] [/f]`; `schtasks /run /tn name`; `schtasks /change /tn name [/enable|/disable] [/tr ...]`; `schtasks /end /tn name`; `schtasks /delete /tn name [/f]`
Key switches	`/sc` once, minute, hourly, daily, weekly, monthly, onlogon, onstart, onidle, onevent; `/st` start time (24h); `/ru SYSTEM`; `/rl highest`; `/xml` export definition
Beginner	`schtasks /query`

Field	Detail
Intermediate	`schtasks /query /fo csv /v > tasks.csv`
Advanced	`schtasks /query /tn "CmdLab\Hello" /xml > hello.xml`
Real-world	Auditing every non-Microsoft task on a server; creating a nightly cleanup task
Output	`/query`: columns Folder, TaskName, Next Run Time, Status. `/v` adds Run As User, Task To Run, Last Result, Author, Schedule Type, Start Time etc.
Exit / Errors	0 success; 1 failure. `ERROR: Access is denied.`; `ERROR: The system cannot find the file specified.` (task name wrong). **Last Result** values: `0x0` success, `0x1` the program returned 1, `0x41301` currently running, `0x41303` has not run yet, `0x41306` terminated by the user, `0x80070002` file not found.
Admin	Not for your own tasks running as you; yes for SYSTEM tasks or other users
Versions	Same on 10/11; `/tr` length limit 261 characters; `/create` times use 24-hour `HH:mm`
Mistakes	Quoting inside `/tr` (nested quotes need `\"`); forgetting `/f` overwrites prompt; leaving test tasks behind; tasks running as SYSTEM with commands pointing to user-writable paths
Security	Suspicious traits: odd or random names; non-Microsoft **Author** in the Microsoft folder; actions in `%TEMP%`, `AppData`, `ProgramData`, `Users\Public`; `powershell`/`cmd`/`mshta`/`wscript` with long or encoded arguments; **hidden** setting; SYSTEM principal; logon/startup triggers on a workstation; creation shortly before an incident. Preserve evidence (export XML) **before** deleting.
Lab / Cleanup	Labs 99-100
Related	`at` (removed), `sc`, `reg query ...TaskCache`, `wevtutil`
PowerShell	`Get-ScheduledTask`, `Register-ScheduledTask`, `Start-ScheduledTask`, `Export-ScheduledTask`

31.2 Investigating a suspicious task

```
1. Identify:      schtasks /query /fo csv /v  -> filter by Author, Task To Run, Run As User
2. Preserve:      schtasks /query /tn "X" /xml > X.xml ; copy the file from %WINDIR%\System32\Tasks
3. Inspect:       who created it (Author), when (Date in XML), what it runs, what account
4. Verify target: does the file exist? signature? hash? location writable by non-admins?
5. Correlate:     TaskScheduler/Operational 106 and 200/201; Security 4698; process creation
6. Contain:       schtasks /change /tn "X" /disable    (reversible)
7. Remove:        schtasks /delete /tn "X" /f          (after evidence is saved)
8. Verify:        query again; check TaskCache registry and Tasks folder for leftovers
9. Learn:         how did it get created? fix that entry point
```

A task deleted through the normal API disappears from both the file store and registry cache; tampered or partially removed tasks may leave one without the other, which is itself a finding.

Lab 99. Create, run, inspect and delete a benign task

Objective. See the lifecycle of a task using a harmless action. **Difficulty.** [I]. **Prerequisites.** Chapter 31. **Environment. VM**, standard prompt (runs as you).

Commands.

```
schtasks /create /tn "CmdLab\Hello" /tr "cmd /c echo task ran >> C:\Users\Public\cmdlab_task.txt" /sc once /st 23
schtasks /query /tn "CmdLab\Hello" /v /fo list
schtasks /run /tn "CmdLab\Hello"
timeout /t 3 /nobreak >nul
type C:\Users\Public\cmdlab_task.txt
schtasks /query /tn "CmdLab\Hello" /fo list /v | findstr /i /c:"Last Result"
schtasks /delete /tn "CmdLab\Hello" /f
schtasks /query /tn "CmdLab\Hello"
del C:\Users\Public\cmdlab_task.txt
```

Expected result. `SUCCESS: The scheduled task "CmdLab\Hello" has successfully been created.`; the verbose query shows `Task To Run`, `Run As User` (you), `Schedule Type: One Time Only`; the file contains `task ran`; `Last Result: 0`; after delete the final query says the task cannot be found. **Explanation.** `/run` triggers on demand regardless of the schedule; the task is only a registered action. **Verification.** `type` output. **Troubleshooting.** If `Last Result` is `0x41303`, it has not run yet; allow a few seconds. If the start time is already past, a warning appears but the task still works with `/run`. **Cleanup.** The delete step. **Challenge.** Disable then re-enable it. **Solution.** `schtasks /change /tn "CmdLab\Hello" /disable` then `/enable`.

Lab 100. Task audit and evidence preservation

Objective. Find non-Microsoft tasks, export one as evidence, and contain it. **Difficulty.** [A]. **Prerequisites.** Lab 99. **Environment. VM**, elevated prompt.

Commands.

```
schtasks /create /tn "CmdLab\Suspect" /tr "cmd /c echo benign >> C:\Users\Public\cmdlab_task.txt" /sc onlogon /f
schtasks /query /fo csv /v > alltasks.csv
findstr /v /i "\\Microsoft\\" alltasks.csv | findstr /i "CmdLab"
schtasks /query /tn "CmdLab\Suspect" /xml > suspect.xml
type suspect.xml | findstr /i "Author Command Arguments LogonTrigger"
schtasks /change /tn "CmdLab\Suspect" /disable
schtasks /query /tn "CmdLab\Suspect" /fo list | findstr /i "Status State"
schtasks /delete /tn "CmdLab\Suspect" /f
```

Expected result. The CSV lists the new task outside the Microsoft folder; the XML shows the author (you), the command and a logon trigger; the task becomes `Disabled`, then is deleted. **Explanation.** This is the contain-then-remove sequence from Section 31.2, performed on a harmless task. **Verification.** `schtasks /query /tn "CmdLab\Suspect"` reports not found. **Troubleshooting.** Logon-trigger tasks generally need elevation to create, hence the elevated prompt. If `findstr` finds nothing in `suspect.xml`, the file may be Unicode; open it in Notepad or run `type suspect.xml` and read it directly. **Cleanup.** `del alltasks.csv suspect.xml C:\Users\Public\cmdlab_task.txt 2>nul`. **Challenge.** Compare the number of non-Microsoft tasks on two machines. **Solution.** Run the audit lines on each and count with `find /c /v ""`.

Part XIII - Windows Networking

From cables and radio to sockets: how a Windows machine gets an address, finds names, routes packets and talks to services, and how to diagnose each layer from CMD.

Chapter 32. Networking Foundations

32.1 The layers you actually troubleshoot

```
Application   browser, ssh, SMB client         <- ports, names, proxy
    |
Transport     TCP (reliable) / UDP (datagram)  <- ports, sockets
    |
Network       IPv4 / IPv6, routing, ICMP       <- addresses, gateway, routes
    |
Link          Ethernet / Wi-Fi, MAC, ARP       <- adapter, switch/AP
    |
Physical      cable, radio                      <- link light, signal
```

Rule of thumb: **test from the bottom up**, and stop at the first layer that fails. A DNS failure is irrelevant if the adapter has no address.

32.2 Core vocabulary

Term	Meaning
MAC address	48-bit hardware address of an adapter, used only on the local link (e.g. `3C-22-FB-12-34-56`)
IPv4 address	32-bit address in dotted decimal (`192.168.1.20`)
IPv6 address	128-bit address in hex groups (`fe80::1234:...` is link-local)
Subnet mask / prefix	Defines which addresses are "on-link": `255.255.255.0` = `/24`
Default gateway	Router used for anything not on-link
DHCP	Protocol that hands out address, mask, gateway, DNS and lease time
APIPA	`169.254.x.x` self-assigned address when DHCP fails: a symptom, not a configuration
DNS	Name-to-address service (UDP/TCP 53)
ARP	Resolves an IPv4 address on the local link to a MAC address
NAT	Router rewrites private source addresses to a public one; explains why inbound connections fail by default
Port	16-bit number identifying a service within an address (TCP 443 = HTTPS)
Socket	Address plus port plus protocol endpoint
MTU	Largest packet on a link (Ethernet usually 1500 bytes)
Proxy	An intermediary for application traffic; configured separately from the adapter

Private IPv4 ranges (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. `127.0.0.0/8` is loopback (this machine); `::1` is IPv6 loopback.

32.3 TCP versus UDP, and socket states

TCP opens a connection with a three-way handshake (SYN, SYN-ACK, ACK), carries a reliable ordered stream, and closes it with FIN. UDP just sends datagrams. Socket states seen in `netstat`:

State	Meaning
`LISTENING`	A program waits for incoming connections on that port
`ESTABLISHED`	Open connection
`TIME_WAIT`	Recently closed, kept briefly (normal, many of them is normal)
`SYN_SENT`	We sent a SYN and see no reply (blocked, down, or filtered)
`CLOSE_WAIT`	Peer closed; local program has not closed yet (application bug if many persist)

A socket bound to `0.0.0.0` (or `[::]`) listens on **all** interfaces; `127.0.0.1` listens on this machine only. That difference matters enormously for exposure.

32.4 How a packet leaves your machine

```
1. App asks for  www.example.com  -> DNS resolves it to an IP
2. Is the IP on my subnet?
      yes -> ARP for its MAC, send directly
      no  -> send to default gateway's MAC (ARP for the gateway)
3. Router forwards hop by hop, NAT rewrites source address at the edge
4. Reply returns along the NAT mapping
```

Each step maps to a tool: `nslookup` (1), `arp -a` and `route print` (2), `tracert` and `pathping` (3), `netstat` (the sockets involved).

32.5 Modern alternatives and deprecated tools

Legacy	Status	Modern
`telnet`	Optional feature, disabled by default; clear-text	`Test-NetConnection host -Port n` (PowerShell), OpenSSH
`ftp`	Present, clear-text	`sftp`, `curl`, PowerShell
`netsh`	Still present and supported; some contexts deprecated	`Get-Net*` PowerShell cmdlets (`Get-NetIPConfiguration`, `Get-NetTCPConnection`, `Get-NetFirewallRule`)
`netsh winsock` / `int ip reset`	Present	Same; both are repair operations
`wmic nic`	Deprecated	`Get-NetAdapter`
`arp`, `route`, `ipconfig`, `nslookup`, `ping`, `tracert`, `pathping`, `netstat`, `getmac`	Present, supported	Keep using

`curl.exe` ships with Windows 10 1803 and later; `curl -I https://example.com` is a quick application-layer check.

Chapter 33. Networking Command Cards

IPCONFIG [External, IP configuration]

Field	Detail
Purpose / Why	Displays and manages the IP configuration of adapters and the DNS client cache
Syntax	`ipconfig [/all] [/release] [/renew] [/flushdns] [/displaydns] [/registerdns]`
Key switches	`/all` full detail (MAC, DHCP server, lease, DNS servers), `/release` and `/renew` give up and request a DHCP lease, `/flushdns` clears the resolver cache, `/displaydns` shows the cache
Beginner	`ipconfig`
Intermediate	`ipconfig /all`
Advanced	`ipconfig /displaydns | findstr /i "Record Name"`
Real-world	First command for any network ticket: does the adapter have an address, gateway and DNS?
Output	Per adapter: media state, connection-specific suffix, IPv4/IPv6 addresses, subnet mask, default gateway. `/all` adds description, physical address, DHCP enabled, lease obtained/expires, DHCP server, DNS servers.
Exit / Errors	0; `/release`/`/renew` errors: `The operation failed as no adapter is in the state permissible for this operation.` Media disconnected means no link.
Admin	`/release`, `/renew`, `/flushdns`, `/registerdns` need elevation; display does not
Versions	Same on 10/11
Mistakes	Releasing the lease over a remote session cuts you off; reading `169.254.x.x` as normal
Security	The DNS cache reveals recently resolved names (useful evidence, also a privacy leak)
Lab / Cleanup	Labs 101-102
Related	`netsh interface ip`, `getmac`, `nslookup`
PowerShell	`Get-NetIPConfiguration`, `Get-NetIPAddress`, `Clear-DnsClientCache`, `Get-DnsClientCache`

PING [External, reachability]

Field	Detail
Purpose / Why	Sends ICMP echo requests to test reachability and round-trip time
Syntax	`ping [-t] [-n count] [-l size] [-f] [-i ttl] [-w ms] [-4|-6] target`
Key switches	`-n` count, `-t` continuous (Ctrl+C stops, Ctrl+Break prints stats), `-l` payload size, `-f` don't fragment, `-4`/`-6` force IP version
Beginner	`ping 127.0.0.1`
Intermediate	`ping -n 4 8.8.8.8`

Field	Detail
Advanced	`ping -f -l 1472 example.com` (path MTU probe: 1472 + 28 header bytes = 1500)
Real-world	The bottom-up ladder: loopback, own IP, gateway, remote IP, remote name
Output	`Reply from IP: bytes=32 time=12ms TTL=117`, then packet loss and min/max/average
Exit / Errors	0 if at least one reply (note: `Destination host unreachable` replies from a router can still return 0 on some builds; read the text); `Request timed out.` no reply; `General failure.` adapter problem; `Ping request could not find host x` DNS failure
Admin	No
Versions	Same
Mistakes	Treating "no reply" as "host down": firewalls routinely block ICMP. Windows Firewall blocks inbound echo by default on Public profile.
Security	ICMP can be filtered or rate-limited; a successful ping proves only that ICMP got through
Lab / Cleanup	Lab 101
Related	`pathping`, `tracert`, `Test-NetConnection`
PowerShell	`Test-Connection`, `Test-NetConnection -ComputerName x -Port 443`

TRACERT and PATHPING [External, path discovery]

Field	Detail
Purpose / Why	`tracert` lists routers along the path using increasing TTL; `pathping` adds per-hop loss statistics over time
Syntax	`tracert [-d] [-h maxhops] [-w ms] target` ; `pathping [-n] [-q queries] [-p ms] [-h hops] target`
Key switches	`-d`/`-n` do not resolve names (faster), `-h` max hops
Beginner	`tracert -d example.com`
Intermediate	`tracert -d -h 15 8.8.8.8`
Advanced	`pathping -n -q 10 8.8.8.8` (takes about 25 seconds per hop)
Real-world	Locating where latency or loss starts: your LAN, ISP, or beyond
Output	One row per hop: three round-trip times and the router; `*` = no response to that probe
Exit / Errors	`Request timed out.` rows are common and not necessarily a fault: many routers deprioritize or drop probe replies
Admin	No
Versions	Same
Mistakes	Concluding a hop with `*` is broken when later hops reply (it is not forwarding-broken); loss on one hop that does not continue to later hops is rate limiting
Security	Reveals network topology; useful reconnaissance, so some networks filter it
Lab / Cleanup	Lab 103
Related	`ping`, `route print`

Field	Detail
PowerShell	`Test-NetConnection -TraceRoute`

NSLOOKUP [External, DNS queries]

Field	Detail
Purpose / Why	Queries DNS servers directly, bypassing the application's resolver path
Syntax	`nslookup [-type=A|AAAA|MX|NS|TXT|CNAME|SOA|PTR] name [server]` ; interactive: `nslookup` then `server x`, `set type=mx`
Beginner	`nslookup example.com`
Intermediate	`nslookup -type=MX example.com`
Advanced	`nslookup example.com 1.1.1.1` (ask a specific server to compare with your default)
Real-world	Telling "DNS server is wrong" from "DNS server is unreachable" from "name does not exist"
Output	The server used (name and address), then `Non-authoritative answer:` and records. `*** can't find x: Non-existent domain` = NXDOMAIN. `DNS request timed out` = server unreachable.
Exit / Errors	Returns 0 even on lookup failure in many builds; **read the text**, not the exit code
Admin	No
Versions	Same; reports the first configured DNS server; `UnKnown` for the server name means no reverse record for its IP (harmless)
Mistakes	Concluding "DNS is fine" because `nslookup` works while the browser fails: browsers may use DNS-over-HTTPS, proxies, hosts file or cache that `nslookup` ignores. `nslookup` does **not** use the hosts file or the system DNS cache.
Security	Resolving through unexpected servers can reveal DNS hijacking; the hosts file (`%WINDIR%\System32\drivers\etc\hosts`) can silently override names; compare `ping name` with `nslookup name`
Lab / Cleanup	Labs 104-105
Related	`ipconfig /displaydns`, `ping`, `type hosts`
PowerShell	`Resolve-DnsName`

NETSTAT [External, sockets and statistics]

Field	Detail
Purpose / Why	Lists listening sockets and connections, statistics, and routing table
Syntax	`netstat [-a] [-n] [-o] [-b] [-e] [-s] [-r] [-p proto] [interval]`
Key switches	`-a` all (listening and connected), `-n` numeric (no name lookups; much faster), `-o` owning PID, `-b` owning executable (**admin; slow**), `-s` per-protocol statistics, `-r` route table, `-p tcp` filter
Beginner	`netstat -an`
Intermediate	`netstat -ano | findstr LISTENING`

Field	Detail
Advanced	`netstat -ano | findstr ":443 "` then `tasklist /fi "pid eq 1234"` (map a socket to a process)
Real-world	"What is listening on this server?" and "Which process owns this connection?"
Output	Proto, Local Address, Foreign Address, State, PID. `0.0.0.0:445` listens on all IPv4 interfaces; `127.0.0.1:5000` only locally.
Exit / Errors	0
Admin	`-b` needs elevation
Versions	Same
Mistakes	Forgetting `-n` (long delays doing reverse DNS); assuming `ESTABLISHED` to a remote IP is malicious
Security	Listing listeners and their PIDs is a core audit step; unexpected listeners on `0.0.0.0`, outbound connections from unusual processes, and many connections to one external address deserve explanation. `netstat` is a point-in-time view: short connections vanish.
Lab / Cleanup	Labs 106-107
Related	`tasklist`, `netsh advfirewall`, Sysinternals TCPView
PowerShell	`Get-NetTCPConnection`, `Get-NetUDPEndpoint`

ARP, ROUTE and GETMAC [External, local network tables]

Field	Detail
Purpose / Why	`arp`: IP-to-MAC cache. `route`: routing table. `getmac`: adapter MAC addresses
Syntax	`arp -a [ip]`, `arp -d [ip]` ; `route print`, `route add dest mask m gateway g [metric n] [-p]`, `route delete dest` ; `getmac [/v] [/fo csv]`
Beginner	`arp -a` ; `getmac /v`
Intermediate	`route print -4`
Advanced	`route print | findstr " 0.0.0.0"` (find the default route(s) and metrics)
Real-world	Diagnosing "can reach the LAN but not the Internet" (missing or wrong default route) or duplicate-IP conflicts (two MACs for one IP)
Output	`arp`: IP, physical address, type (dynamic/static). `route print`: interface list and IPv4/IPv6 route tables; **the lowest metric wins**.
Exit / Errors	`The requested operation requires elevation.` for add/delete
Admin	`route add/delete`, `arp -d` need elevation
Versions	Same
Mistakes	`route add` without `-p` vanishes on reboot (intentional for tests); a wrong persistent route (`-p`) can break connectivity until removed
Security	Static ARP/route changes can redirect traffic; unexpected persistent routes are a finding
Lab / Cleanup	Lab 108
Related	`ipconfig`, `netsh interface ip`

Field	Detail
PowerShell	`Get-NetNeighbor`, `Get-NetRoute`, `New-NetRoute`, `Get-NetAdapter`

NETSH [External, network shell]

Field	Detail
Purpose / Why	Context-based tool for configuring network components: interfaces, Wi-Fi, firewall, proxy, Winsock
Syntax	`netsh <context> <subcontext> <command>` ; `netsh /?` ; interactive: type `netsh`
Key switches	`netsh interface ip show config`, `netsh interface ipv4 show subinterfaces`, `netsh wlan show interfaces`, `netsh wlan show profiles`, `netsh advfirewall show allprofiles`, `netsh winhttp show proxy`, `netsh trace`
Beginner	`netsh interface ip show config`
Intermediate	`netsh wlan show interfaces` (SSID, signal %, channel, radio type)
Advanced	`netsh winhttp show proxy` (system-level proxy used by some services)
Real-world	Reading Wi-Fi diagnostics; auditing the firewall; checking proxy settings that break services
Output	Context-specific text
Exit / Errors	`The following command was not found: ...`; `The requested operation requires elevation (Run as administrator).`
Admin	Display often no; changes yes
Versions	Parts are deprecated in favor of PowerShell; `netsh` itself remains supported
Mistakes	Resets (`netsh winsock reset`, `netsh int ip reset`) are **repair operations that discard custom network configuration and need a reboot**; do not run them as a first step. Note that `netsh wlan show profile ... key=clear` can display stored Wi-Fi passphrases to an administrator; treat such output as a secret.
Security	Firewall rule and proxy changes made through `netsh` appear in logs when auditing is on; unexpected proxy settings can redirect application traffic
Lab / Cleanup	Labs 109-110
Related	PowerShell `Get-Net*` cmdlets
PowerShell	`Get-NetFirewallProfile`, `Get-NetAdapter`, `Get-NetIPInterface`

Chapter 34. Troubleshooting Decision Trees

34.1 The bottom-up ladder

```
[1] ipconfig /all
      no adapter / "Media disconnected" ........ -> physical or Wi-Fi problem (cable, AP, adapter enabled?)
      IPv4 = 169.254.x.x ....................... -> DHCP failure (server, VLAN, lease); try ipconfig /release the
      has valid IP, mask, gateway .............. -> go on
[2] ping 127.0.0.1
      fail ..................................... -> TCP/IP stack problem (rare); netsh int ip reset (last resort)
[3] ping <my own IP>
[4] ping <default gateway>
      fail ..................................... -> local link: wrong VLAN/subnet, switch/AP, duplicate IP (arp -
[5] ping 8.8.8.8  (or any known public IP)
      fail ..................................... -> routing/NAT/ISP problem: tracert -d 8.8.8.8 shows where it st
[6] ping example.com
      "could not find host" .................... -> DNS problem -> Section 34.2
      replies .................................. -> network layer is healthy
[7] test the port: curl -I https://example.com  /  Test-NetConnection host -Port 443
      port fails, ping works ................... -> firewall, wrong port, service not listening
      works but browser fails .................. -> proxy, certificate/time, browser DNS-over-HTTPS
```

34.2 "The browser works but DNS commands fail" (and its mirror)

Walk through step by step:

1. **Compare the paths.** `ping example.com` uses the system resolver (cache, hosts file, DNS client service). `nslookup example.com` queries the configured DNS server directly. They can disagree.
2. If `nslookup` fails but the browser works: the browser may be using **DNS-over-HTTPS** (a built-in secure resolver), a **proxy** (the proxy resolves names), or **cached** results. Check: `ipconfig /displaydns`, `netsh winhttp show proxy`, browser DoH settings.
3. If `nslookup` works but `ping name` fails: the **hosts file** has a bad entry, the **DNS Client** service is stopped (`sc query Dnscache`), a stale cache entry exists (`ipconfig /flushdns`), or a **NRPT/VPN split-DNS** policy sends the name elsewhere.
4. If `nslookup` shows `can't find ... Non-existent domain`: the name does not exist **on that server**. Ask another (`nslookup name 1.1.1.1`); if only your server fails, its zone or forwarder is wrong.
5. If `nslookup` shows `DNS request timed out`: the server is unreachable or filtering. `ping <dns-server>`, `tracert -d <dns-server>`, check firewall/VPN.
6. If both work from CMD but the app fails: check IPv4 versus IPv6 (`ping -4`, `ping -6`), proxy settings, and system time (certificate errors look like connectivity errors).

Evidence to record at each step: exact command, exact output, time. Change **one thing at a time**, and re-test.

34.3 Other common scenarios

Symptom	Likely causes	First checks
Wi-Fi connected, no Internet	Captive portal; DHCP/DNS failure; wrong gateway	`ipconfig /all`, open a plain-HTTP page, `ping gateway`
Can reach IP but not names	DNS	`nslookup`, `ipconfig /all` DNS servers, VPN

Symptom	Likely causes	First checks
Can reach LAN but not Internet	Missing default route; gateway down; NAT/ISP	`route print -4`, `ping gateway`, `tracert -d 8.8.8.8`
Intermittent drops	Weak signal, interference, bad cable, MTU problem	`netsh wlan show interfaces`, `ping -t gateway`, `pathping`
Large transfers stall, small work	MTU/fragmentation	`ping -f -l 1472 host`, reduce until it passes
"IP address conflict" popup	Two hosts with same IP	`arp -a` after `ping`; check DHCP reservations
Works on one network, not another	Proxy, firewall profile (Public vs Private), DNS suffix	`netsh winhttp show proxy`, firewall profile, `ipconfig /all`
File sharing fails	SMB blocked (TCP 445), network profile Public, DNS	`Test-NetConnection host -Port 445`, `netstat -ano | findstr :445`

Lab 101. The ping ladder

Objective. Run the bottom-up test in order and record each result. **Difficulty.** [B]. **Prerequisites.** Section 34.1. **Environment.** Any prompt, network connection.

Commands.

```
ipconfig
ping -n 2 127.0.0.1
ping -n 2 %COMPUTERNAME%
ping -n 2 <your-gateway-from-ipconfig>
ping -n 2 8.8.8.8
ping -n 2 example.com
```

Expected result. Replies at every rung; local rungs under 1 ms, gateway a few ms, Internet tens of ms. **Explanation.** A failure points to a specific layer. **Verification.** Write down the first rung that fails if any. **Troubleshooting.** Replace `<your-gateway-from-ipconfig>` with the actual address; some gateways block ping, so absence of replies there is inconclusive if later rungs work. **Cleanup.** None. **Challenge.** Script the ladder with pass/fail output. **Solution.** `for %h in (127.0.0.1 8.8.8.8) do @ping -n 1 %h | find "TTL=" >nul && (echo %h OK) || (echo %h FAIL)`.

Lab 102. Read `ipconfig /all` completely

Objective. Identify every field and decide whether the configuration is healthy. **Difficulty.** [B]. **Prerequisites.** `ipconfig` card. **Environment.** Any prompt.

Commands.

```
ipconfig /all > net_base.txt
type net_base.txt | more
findstr /i "Description Physical IPv4 Subnet Gateway DHCP DNS Lease" net_base.txt
```

Expected result. A line each for description, MAC, IPv4, mask, gateway, DHCP server, DNS servers, lease times for your active adapter. **Explanation.** Link to the vocabulary in Section 32.2. **Verification.** Is the gateway

inside the subnet defined by IP and mask? Is DHCP enabled? **Troubleshooting.** Virtual adapters (VPN, WSL, Hyper-V) add noise; focus on the adapter with a gateway. **Cleanup.** Keep `net_base.txt` for Lab 110. **Challenge.** Compute the network address from IP and mask. **Solution.** AND the octets; for 192.168.1.20/255.255.255.0 it is 192.168.1.0.

Lab 103. Where does the path slow down?

Objective. Use `tracert` and `pathping` to localize latency. **Difficulty.** [I]. **Prerequisites.** Lab 101. **Environment.** Any prompt, Internet access.

Commands.

```
tracert -d example.com
pathping -n -q 5 -p 200 example.com
```

Expected result. A hop list; hop 1 is your gateway; times generally increase with distance. **Explanation.** Compare the biggest jump between consecutive hops, and check whether loss continues to the destination. **Verification.** Does the final hop respond? **Troubleshooting.** Rows of `*` in the middle are common and benign if the destination replies. **Cleanup.** None. **Challenge.** Explain why a lossy hop followed by clean hops is not a problem. **Solution.** The router is rate-limiting probe replies (control-plane), not dropping forwarded traffic.

Lab 104. DNS: resolver versus server

Objective. Show that `nslookup` and the system resolver can disagree, using a safe hosts-file entry. **Difficulty.** [A]. **Prerequisites.** Section 34.2. **Environment. VM**, elevated prompt.

Commands.

```
copy %WINDIR%\System32\drivers\etc\hosts hosts.backup
echo 127.0.0.1 labtest.invalid>> %WINDIR%\System32\drivers\etc\hosts
ping -n 1 labtest.invalid
nslookup labtest.invalid
copy /y hosts.backup %WINDIR%\System32\drivers\etc\hosts
ping -n 1 labtest.invalid
```

Expected result. `ping` resolves `labtest.invalid` to 127.0.0.1 (the hosts entry), `nslookup` reports non-existent domain, and after restore ping fails to find the host. **Explanation.** `nslookup` bypasses the hosts file; the resolver does not. **Verification.** `findstr labtest %WINDIR%\System32\drivers\etc\hosts` shows nothing after restore. **Troubleshooting.** Security software may block edits to the hosts file; use a VM. **Cleanup.** The restore step; `del hosts.backup`. **Challenge.** Explain why attackers and defenders both care about this file. **Solution.** Whoever controls hosts controls name resolution for the machine; integrity monitoring should include it.

Lab 105. Flush, display and compare the DNS cache

Objective. Observe the DNS client cache. **Difficulty.** [I]. **Prerequisites.** Lab 104. **Environment.** Elevated prompt.

Commands.

```
ipconfig /flushdns
ping -n 1 example.com
ipconfig /displaydns | findstr /i "example.com"
ipconfig /flushdns
ipconfig /displaydns | findstr /i "example.com"
```

Expected result. The name appears in the cache after the ping and not after the second flush. **Explanation.** The cache holds recent answers until their TTL expires. **Verification.** The second `findstr` prints nothing. **Troubleshooting.** Some builds hide entries; compare with `Get-DnsClientCache` in PowerShell. **Cleanup.** None. **Challenge.** Find the TTL. **Solution.** `ipconfig /displaydns` lists `Time To Live` for each record.

Lab 106. What is listening here?

Objective. Build a listener inventory. **Difficulty.** [I]. **Prerequisites.** `netstat` card. **Environment.** Elevated prompt preferred.

Commands.

```
netstat -ano | findstr LISTENING > listeners.txt
type listeners.txt
for /f "tokens=5" %p in (listeners.txt) do @tasklist /fi "pid eq %p" /fo csv /nh
```

Expected result. A list of listening sockets with PIDs, then the owning process names. **Explanation.** Maps each open port to the program responsible. **Verification.** Note addresses: `0.0.0.0` or `[::]` (all interfaces) versus `127.0.0.1` (local only). **Troubleshooting.** PID 4 is the Windows kernel (System), which hosts SMB. **Cleanup.** `del listeners.txt`. **Challenge.** List only sockets exposed on all interfaces. **Solution.** `netstat -ano | findstr "0.0.0.0: [::]:" | findstr LISTENING`.

Lab 107. Connect to a local test listener and watch it appear

Objective. Create a harmless listener and observe it in `netstat`. **Difficulty.** [I]. **Prerequisites.** Lab 106. **Environment. VM**, PowerShell available.

Commands. In window A (a one-line listener bound to loopback only):

```
powershell -NoProfile -Command "$l=[Net.Sockets.TcpListener]::new([Net.IPAddress]::Loopback,5555); $l.Start(); '
```

In window B:

```
netstat -ano | findstr :5555
powershell -NoProfile -Command "Test-NetConnection 127.0.0.1 -Port 5555 | Select TcpTestSucceeded"
```

Expected result. `LISTENING` on `127.0.0.1:5555` with the PowerShell PID; `TcpTestSucceeded : True`. **Explanation.** The socket is bound to loopback, so it is not reachable from other machines. **Verification.** `tasklist /fi "pid eq <PID>"` shows `powershell.exe`. **Troubleshooting.** If the port is in use, pick another. **Cleanup.** Ctrl+C in window A; confirm `netstat -ano | findstr :5555` is empty. **Challenge.** Explain how this teaches detection. **Solution.** An unexpected listener is found by port, PID, image path, parent process and command line; here every attribute is known and benign.

Lab 108. Routes and ARP

Objective. Read the routing table and ARP cache; add and remove a harmless temporary route. **Difficulty.** [A]. **Prerequisites.** `route` card. **Environment. VM**, elevated.

Commands.

```
route print -4
arp -a
route add 203.0.113.0 mask 255.255.255.0 <your-gateway> metric 500
route print -4 | findstr 203.0.113
ping -n 1 203.0.113.1
route delete 203.0.113.0
route print -4 | findstr 203.0.113
```

Expected result. The test route (documentation range 203.0.113.0/24, reserved for examples) appears, the ping times out or is unreachable, and the route disappears after delete. **Explanation.** Routes are matched by the most specific prefix, then lowest metric. **Verification.** Last `findstr` prints nothing. **Troubleshooting.** Substitute your real gateway address. **Cleanup.** The delete step (the route was not persistent). **Challenge.** Identify the default route. **Solution.** The entry with destination `0.0.0.0` and mask `0.0.0.0`; if several, the lowest metric is used.

Lab 109. Wi-Fi and adapter diagnostics (read-only)

Objective. Read link quality and adapter data. **Difficulty.** [I]. **Prerequisites.** `netsh` card. **Environment.** Any prompt on a Wi-Fi machine (skip on wired-only).

Commands.

```
netsh wlan show interfaces
netsh interface ipv4 show subinterfaces
getmac /v /fo list
netsh winhttp show proxy
```

Expected result. SSID, signal quality, radio type, channel; MTU per interface; adapter MACs; the system proxy state (normally `Direct access (no proxy server)`). **Explanation.** Link problems appear here before they appear in `ping`. **Verification.** Move closer to the access point and watch the signal change. **Troubleshooting.** `There is no wireless interface` on wired PCs. **Cleanup.** None. **Challenge.** Extract just the signal percentage. **Solution.** `netsh wlan show interfaces | findstr /i "Signal"`.

Lab 110. Network health snapshot script

Objective. Produce a one-file network report with pass/fail checks. **Difficulty.** [A]. **Prerequisites.** Labs 101-109. **Environment.** Scratch folder.

Commands. Save as `netcheck.cmd`:

```
@echo off
setlocal EnableExtensions
set "OUT=%~dp0netcheck.txt"
> "%OUT%" echo Network check on %COMPUTERNAME% at %DATE% %TIME%
>> "%OUT%" echo ---- ipconfig /all
ipconfig /all >> "%OUT%"
>> "%OUT%" echo ---- route
route print -4 >> "%OUT%"
>> "%OUT%" echo ---- tests
for %%h in (127.0.0.1 8.8.8.8) do (
    ping -n 1 -w 1500 %%h | find "TTL=" >nul && (>>"%OUT%" echo PASS ping %%h) || (>>"%OUT%" echo FAIL ping %%h)
)
nslookup example.com 2>&1 | find "Name:" >nul && (>>"%OUT%" echo PASS dns) || (>>"%OUT%" echo FAIL dns)
type "%OUT%" | findstr /b "PASS FAIL"
exit /b 0
```

Expected result. A summary of PASS/FAIL lines and a full report file. **Explanation.** A reusable first-response collector. **Verification.** Disconnect the network and run again: the pass lines flip. **Troubleshooting.** `find "TTL="` and `find "Name:"` rely on English output wording; `nslookup` prints its own server's `Address:` line even on failure, which is why the DNS test looks for the `Name:` line of a successful answer instead. **Cleanup.** `del netcheck.cmd netcheck.txt`. **Challenge.** Add a TCP 443 test. **Solution.** `powershell -NoProfile -Command "(Test-NetConnection example.com -Port 443).TcpTestSucceeded"` and test for `True`.

Part XIV - Windows Firewall

The host firewall: profiles, rules, defaults, logging, safe inspection and controlled changes.

Chapter 35. Windows Defender Firewall from CMD

35.1 Concepts

Windows Defender Firewall with Advanced Security is a stateful, host-based packet filter. It decides which network traffic may enter or leave this computer.

Concept	Meaning
Profile	Domain (joined domain network), Private (trusted), Public (untrusted). The active profile is chosen per network; Public is the most restrictive.
Default policy	Typically **inbound blocked unless allowed**, **outbound allowed unless blocked**, per profile
Rule	Direction (in/out), action (allow/block), protocol, local/remote ports and addresses, program, service, profile, interface type, users/groups
Stateful	Replies to allowed outbound connections are allowed back in automatically
Rule precedence	**Block rules override allow rules**; more specific rules do not automatically beat general ones. Group Policy rules can override local rules depending on merge settings.
Services and programs	Rules can be tied to an executable path or a service SID, which is more precise than a port alone
Logging	Disabled by default; dropped and successful connections can be logged to `%SystemRoot%\System32\LogFiles\Firewall\pfirewall.log`

Firewall events (Microsoft-Windows-Windows Firewall With Advanced Security/Firewall log): 2004 rule added, 2005 rule changed, 2006 rule deleted, 2003 profile setting changed. Security log (needs auditing): 5156/5157 allowed/blocked connection, 5152 packet dropped.

> **SECURITY.** The firewall is a **host control that local administrators can change**. Turning it off, adding broad allow rules or disabling profiles are findings; they are logged (2003/2004/2005/2006) and are standard items in compromise assessments.

35.2 Inspecting safely (read-only)

```
netsh advfirewall show allprofiles
netsh advfirewall show currentprofile
netsh advfirewall firewall show rule name=all dir=in status=enabled
netsh advfirewall firewall show rule name=all dir=in | find /c "Rule Name"
netsh advfirewall export "%TEMP%\fw_backup.wfw"
```

`show allprofiles` prints, for Domain, Private and Public: `State ON/OFF`, `Firewall Policy` (for example `BlockInbound,AllowOutbound`), logging settings. **Export before any change**; restore with `netsh advfirewall import file` (this **replaces** the whole configuration).

35.3 Controlled changes (modify protection)

```
netsh advfirewall firewall add rule name="CmdLab Test Rule" dir=in action=allow protocol=TCP localport=5555 profi
netsh advfirewall firewall show rule name="CmdLab Test Rule"
netsh advfirewall firewall delete rule name="CmdLab Test Rule"
```

Rules for rules: give each a **unique, descriptive name**; restrict by **program, profile and remote address** whenever possible; allow only what the service needs; **always script the delete** for temporary rules; never disable the firewall to "see if it fixes it" on a production host (use a targeted rule or the log instead).

```
netsh advfirewall set currentprofile logging droppedconnections enable
netsh advfirewall set currentprofile logging filename "%SystemRoot%\System32\LogFiles\Firewall\pfirewall.log"
type "%SystemRoot%\System32\LogFiles\Firewall\pfirewall.log" | more
```

(Enabling dropped-connection logging is a configuration change; reverse it with `droppedconnections disable` when the investigation ends. The log lists date, time, action DROP/ALLOW, protocol, source/destination IP, ports, size, TCP flags and direction.)

35.4 Auditing a firewall configuration (defender checklist)

1. All three profiles `State ON`. Anything else is a finding.
2. Default inbound policy is `BlockInbound` on every profile.
3. Enumerate enabled **inbound allow** rules; for each, confirm business need, **program path**, **remote address scope** (not `Any` for sensitive ports), and profile.
4. Find **allow rules for programs in user-writable folders** (`Downloads`, `AppData`, `Temp`, `Users\Public`).
5. Find rules with names that are empty, random, or mimic system names.
6. Export the configuration to a dated file and diff it against the previous baseline.
7. Confirm Group Policy merge behavior if the machine is managed (`netsh advfirewall show global`).
8. Confirm logging is on where required, with adequate log size.

Lab 111. Firewall inspection report

Objective. Capture the firewall posture into files. **Difficulty.** [I]. **Prerequisites.** Section 35.2. **Environment.** Elevated prompt, scratch folder.

Commands.

```
netsh advfirewall show allprofiles > fw_profiles.txt
netsh advfirewall firewall show rule name=all dir=in status=enabled > fw_in_rules.txt
netsh advfirewall export fw_backup.wfw
findstr /i "State Policy" fw_profiles.txt
find /c "Rule Name" fw_in_rules.txt
```

Expected result. Three `State ON` lines, `BlockInbound,AllowOutbound` policies, and a rule count in the dozens or more. **Explanation.** These are the first three artifacts of any firewall audit. **Verification.** Confirm each file is non-empty. **Troubleshooting.** Without elevation, `export` fails. **Cleanup.** Keep `fw_backup.wfw` for Lab 112. **Challenge.** List only rules allowing inbound on `Any` remote address for TCP. **Solution.** `netsh advfirewall firewall show rule name=all dir=in status=enabled verbose` and search for `RemoteIP: Any` with `Protocol: TCP` (PowerShell `Get-NetFirewallRule` is easier).

Lab 112. Add, test and remove a scoped test rule, with rollback

Objective. Make and undo a firewall change safely. **Difficulty.** [A]. **Prerequisites.** Lab 111, Lab 107. **Environment. VM**, elevated prompt.

Commands.

```
netsh advfirewall export fw_before.wfw
netsh advfirewall firewall add rule name="CmdLab Test Rule" dir=in action=allow protocol=TCP localport=5555 profi
netsh advfirewall firewall show rule name="CmdLab Test Rule"
netsh advfirewall firewall delete rule name="CmdLab Test Rule"
netsh advfirewall firewall show rule name="CmdLab Test Rule"
wevtutil qe "Microsoft-Windows-Windows Firewall With Advanced Security/Firewall" /c:5 /rd:true /f:text | findstr
```

Expected result. Rule added and displayed with `Direction In`, `Action Allow`, `Profiles Private`, `RemoteIP LocalSubnet`; after delete `No rules match the specified criteria.`; the log shows events 2004 and 2006 for the rule (if the log is enabled). **Explanation.** The rule is scoped to one port, one profile and the local subnet, which is the pattern to copy. **Verification.** The final show command. **Troubleshooting.** The event log path name may vary by build; list logs with `wevtutil el | find /i "firewall"`. **Cleanup.** If anything looks wrong: `netsh advfirewall import fw_before.wfw`, then `del fw_before.wfw fw_backup.wfw fw_profiles.txt fw_in_rules.txt`. **Challenge.** Explain how a defender would find this rule later if it had been left behind. **Solution.** Enumerate enabled inbound rules, filter by name and creation event 2004, inspect program/port/scope, and compare with the exported baseline.

Part XV - Windows Event Logging

The system's flight recorder: channels, providers, event IDs, XML structure, querying and creating events, and what to look for.

Chapter 36. Event Log Architecture and wevtutil

36.1 Architecture

Windows records events through the **Windows Event Log service**. Applications and components are **providers** (publishers) that write **events** into **channels**; a channel is stored as an `.evtx` file under `%SystemRoot%\System32\winevt\Logs`.

Concept	Meaning
Provider	The component writing events (`Microsoft-Windows-Security-Auditing`, `Service Control Manager`)
Channel / log	A named stream: classic logs **Application**, **Security**, **System** (plus Setup, Forwarded Events) and many **Applications and Services Logs** such as `Microsoft-Windows-TaskScheduler/Operational`, `Microsoft-Windows-Windows Defender/Operational`, `Microsoft-Windows-PowerShell/Operational`
Event ID	A number identifying the *kind* of event **within its provider**; the same number can mean different things in different logs
Level	1 Critical, 2 Error, 3 Warning, 4 Information, 5 Verbose
Keywords / Task / Opcode	Further categorization (Security log: Audit Success or Audit Failure)
Channel types	Admin, Operational, Analytic, Debug (analytic/debug are off by default and not for routine use)

Reading the **Security** log requires administrator rights (or membership in *Event Log Readers*), and **auditing must be enabled** for most security events to exist at all.

36.2 Event XML structure

Every event is an XML document; `wevtutil ... /f:xml` shows it.

```
<Event>
  <System>
    <Provider Name="Service Control Manager" Guid="{...}" />
    <EventID>7045</EventID>
    <Version>0</Version> <Level>4</Level> <Task>0</Task> <Opcode>0</Opcode> <Keywords>0x8080000000000000</Keyword
    <TimeCreated SystemTime="2026-10-01T09:15:42.123456700Z" />   <- UTC, not local
    <EventRecordID>12345</EventRecordID>                           <- sequence number in this log
    <Execution ProcessID="740" ThreadID="1840" />                  <- process that wrote the event
    <Channel>System</Channel>
    <Computer>HOST01</Computer>
    <Security UserID="S-1-5-18" />                                 <- SID of the writing account
  </System>
  <EventData>                                                     <- event-specific fields
    <Data Name="ServiceName">CmdLabSvc</Data>
    <Data Name="ImagePath">C:\Windows\System32\cmd.exe</Data>
    <Data Name="ServiceType">user mode service</Data>
    <Data Name="StartType">demand start</Data>
    <Data Name="AccountName">LocalSystem</Data>
  </EventData>
</Event>
```

Investigation reminders: **timestamps are UTC** in XML (text format converts to local); record IDs reveal **gaps** (deleted events); the `Execution ProcessID` is the *logging* process, not necessarily the subject of the event; event-specific facts live in `EventData`.

36.3 Important events (reference)

Log	Event ID	Meaning
Security	4624 / 4625	Successful / failed logon (fields: Logon Type, account, source IP, process)
Security	4634, 4647	Logoff, user-initiated logoff
Security	4648	Logon with explicit credentials (runas, scheduled tasks)
Security	4672	Special privileges assigned to a new logon (admin-level logon)
Security	4688	Process created (command line only if the policy "Include command line in process creation events" is on)
Security	4697 / System 7045	Service installed
Security	4698, 4699, 4700, 4701, 4702	Scheduled task created, deleted, enabled, disabled, updated
Security	4719	Audit policy changed
Security	4720, 4722, 4724, 4726	User created, enabled, password reset attempt, deleted
Security	4732, 4733	Member added to / removed from a local security group
Security	4657	Registry value modified (needs SACL auditing)
Security	1102	**Security log cleared**
System	104	**A log was cleared** (System or other logs)
System	7036 / 7040	Service entered running/stopped state / service start type changed
System	6005 / 6006 / 6008	Event log started (boot) / stopped (clean shutdown) / unexpected shutdown
System	41	Kernel-Power: rebooted without clean shutdown
TaskScheduler/Operational	106, 140, 141, 200, 201	Task registered, updated, deleted, action started, action completed (history must be enabled)
Defender/Operational	1116, 1117	Malware detected, protective action taken
Defender/Operational	5001, 5007	Real-time protection disabled, configuration changed
Windows Firewall/Firewall	2003 to 2006	Profile/rule changes
PowerShell/Operational	4104	Script block logged (when enabled)

Logon types in 4624/4625: 2 interactive (keyboard), 3 network (file share), 4 batch (scheduled task), 5 service, 7 unlock, 8 network cleartext, 9 new credentials (runas /netonly), 10 remote interactive (Remote Desktop), 11 cached interactive. Logon type 3 or 10 from an unexpected source IP deserves attention.

> **NOTE.** The auditing policy decides whether most Security events exist. Inspect it with `auditpol /get /category:*` (read-only, elevated). Changing it with `auditpol /set` is a configuration change that belongs in a documented policy.

WEVTUTIL [External, event logs]

Field	Detail
Purpose / Why	Enumerate logs, query events, export and archive logs, change log settings
Syntax	`wevtutil el` ; `wevtutil gl log` ; `wevtutil gli log` ; `wevtutil qe log [/q:"xpath"] [/c:n] [/rd:true] [/f:text|xml|rendered] [/lf:true]` ; `wevtutil epl log file.evtx [/q:...]` ; `wevtutil sl log [/e:true|false] [/ms:bytes]` ; `wevtutil cl log`
Key switches	`/c:5` count, `/rd:true` newest first, `/f:text` readable, `/f:xml` complete, `/q` XPath filter, `/lf:true` treat the log argument as an `.evtx` file path
Beginner	`wevtutil el`
Intermediate	`wevtutil qe System /c:5 /rd:true /f:text`
Advanced	`wevtutil qe Security "/q:*[System[(EventID=4625)]]" /c:10 /rd:true /f:text` ; last 24 hours: `"/q:*[System[TimeCreated[timediff(@SystemTime) <= 86400000]]]"`
Real-world	Pulling the logon history around an incident; exporting logs as evidence
Output	Text blocks per event (`Event[0]`, `Log Name`, `Source`, `Date`, `Event ID`, `Level`, `User`, `Computer`, `Description`) or XML
Exit / Errors	5 access denied (Security without admin); 15007 channel not found; 15011 invalid query
Admin	Security log and settings changes: yes
Versions	Present since Vista; `Get-WinEvent` is the PowerShell equivalent and handles filtering more comfortably
Mistakes	Forgetting quotes around the `/q:` expression (parentheses, asterisks); expecting local time in XML; `wevtutil cl` **destroys evidence**
Security	**Clearing logs is a high-signal event (1102/104) and should happen only through documented procedure after export.** Export (`epl`) before any clear. Attackers clear logs; defenders forward logs off-box (Windows Event Forwarding, SIEM) so local clearing does not erase history.
Lab / Cleanup	Labs 151-155
Related	`eventcreate`, `auditpol`, `Get-WinEvent`
PowerShell	`Get-WinEvent -LogName Security -FilterHashtable @{Id=4625}`, `Get-EventLog` (legacy)

EVENTCREATE [External, write events]

Field	Detail
Purpose / Why	Writes a custom event to a log; useful for scripts to leave an audit trail
Syntax	`eventcreate /t error|warning|information /id 1-1000 /l application|system /so source /d "description"`
Beginner	`eventcreate /t information /id 100 /l application /so CMDLAB /d "hello from CMD"`
Intermediate	Log script milestones with distinct IDs: 100 start, 101 success, 900 failure

Field	Detail
Advanced	Combine with `wevtutil qe Application "/q:*[System[Provider[@Name='CMDLAB']]]" /f:text`
Real-world	Making an unattended maintenance script visible to the central monitoring system
Output	`SUCCESS: An event of type 'INFORMATION' was created in the 'application' log/source 'CMDLAB'.`
Exit / Errors	`ERROR: Access is denied.`; invalid ID (must be 1 to 1000)
Admin	Yes
Versions	Same
Mistakes	Putting secrets or full command lines containing secrets in the description; inventing sources that collide with real ones
Security	Anyone with admin rights can write misleading events, so custom events are convenience records, not tamper-proof evidence
Lab / Cleanup	Lab 153; remove the registered source afterward
Related	`wevtutil`, `logger` equivalents
PowerShell	`Write-EventLog`, `New-EventLog`

Lab 151. Discover logs and read their configuration

Objective. List channels and see size, retention and status. **Difficulty.** [B]. **Prerequisites.** Section 36.1. **Environment.** Standard prompt (elevated for Security).

Commands.

```
wevtutil el | find /c /v ""
wevtutil el | findstr /i "TaskScheduler Defender Firewall"
wevtutil gli System
wevtutil gl System
```

Expected result. A few hundred channels; the three named families; creation/last-write times, record counts; `enabled: true`, `maxSize`, `retention: false` (overwrite as needed). **Explanation.** Size and retention decide how far back evidence goes. **Verification.** Compare `numberOfLogRecords` with the oldest event date. **Troubleshooting.** `Access is denied` for Security needs elevation. **Cleanup.** None. **Challenge.** Find the oldest System event. **Solution.** `wevtutil qe System /c:1 /rd:false /f:text`.

Lab 152. Logon history and failed logons

Objective. Extract logons and failures with an XPath filter. **Difficulty.** [I]. **Prerequisites.** Section 36.3. **Environment. VM**, elevated.

Commands.

```
wevtutil qe Security "/q:*[System[(EventID=4624)]]" /c:5 /rd:true /f:text | findstr /i "Date: Logon Account Sourc
runas /user:%COMPUTERNAME%\nosuchuser cmd
wevtutil qe Security "/q:*[System[(EventID=4625)]]" /c:3 /rd:true /f:text
```

Expected result. Recent successful logons (types 2, 5, 7, 11 are typical on a desktop) and, usually, a 4625 for the failed `runas` attempt. **Explanation.** Failed and successful logons are the backbone of intrusion review. **Verification.** The failure event names the account `nosuchuser`. **Troubleshooting.** Empty results mean logon auditing is off: check `auditpol /get /category:"Logon/Logoff"`. If no 4625 appears for a nonexistent account, repeat with a real test account and a deliberately wrong password. **Cleanup.** None. **Challenge.** Count 4625 events in the last hour. **Solution.** `wevtutil qe Security "/q:*[System[(EventID=4625) and TimeCreated[timediff(@SystemTime) <= 3600000]]]" /f:text | find /c "Event ID"`.

Lab 153. Write and find your own event

Objective. Create a custom event and retrieve it. **Difficulty.** [I]. **Prerequisites.** `eventcreate` card. **Environment. VM**, elevated.

Commands.

```
eventcreate /t information /id 100 /l application /so CMDLAB /d "lab event one"
eventcreate /t warning /id 200 /l application /so CMDLAB /d "lab event two"
wevtutil qe Application "/q:*[System[Provider[@Name='CMDLAB']]]" /f:text /rd:true
wevtutil qe Application "/q:*[System[Provider[@Name='CMDLAB'] and (EventID=200)]]" /f:xml
```

Expected result. Two events with the right IDs and levels; the XML shows `Provider Name='CMDLAB'` and `EventData`. **Explanation.** Scripts can leave their own audit trail. **Verification.** Note the `TimeCreated` in UTC. **Troubleshooting.** The `eventcreate` ID must be 1-1000. **Cleanup.** `reg delete HKLM\SYSTEM\CurrentControlSet\Services\EventLog\Application\CMDLAB /f` (removes the lab source registration). **Challenge.** Log success/failure from the Section 21.3 skeleton. **Solution.** Add `eventcreate` calls inside `:log` for ERROR level only.

Lab 154. Export and examine a log offline

Objective. Export a filtered log and read it as a file. **Difficulty.** [I]. **Prerequisites.** `wevtutil` card. **Environment. VM**, elevated.

Commands.

```
wevtutil epl System sys_copy.evtx "/q:*[System[(EventID=7036)]]"
wevtutil qe sys_copy.evtx /lf:true /c:3 /f:text
dir sys_copy.evtx
```

Expected result. A new `.evtx` file containing only service state-change events; the query prints three of them. **Explanation.** Exported files can be analyzed elsewhere without touching the live log. **Verification.** `/lf:true` tells `wevtutil` the argument is a file path. **Troubleshooting.** Quote the XPath. **Cleanup.** `del sys_copy.evtx`. **Challenge.** Archive a log with its message resources for another machine. **Solution.** `wevtutil al archive.evtx` creates a `LocaleMetaData` folder next to the file.

Lab 155. Task Scheduler history and service-install events

Objective. Enable history, perform a benign action, and find the evidence. **Difficulty.** [A]. **Prerequisites.** Labs 87, 99. **Environment. VM + snapshot**, elevated.

Commands.

```
wevtutil sl Microsoft-Windows-TaskScheduler/Operational /e:true
schtasks /create /tn "CmdLab\Evidence" /tr "cmd /c echo evidence >> C:\Users\Public\cmdlab_task.txt" /sc once /st
schtasks /run /tn "CmdLab\Evidence"
sc create CmdLabSvc binPath= "%WINDIR%\System32\cmd.exe" start= demand
wevtutil qe Microsoft-Windows-TaskScheduler/Operational /c:10 /rd:true /f:text | findstr /i "Event ID: Task"
wevtutil qe System "/q:*[System[(EventID=7045)]]" /c:1 /rd:true /f:text
sc delete CmdLabSvc
schtasks /delete /tn "CmdLab\Evidence" /f
wevtutil sl Microsoft-Windows-TaskScheduler/Operational /e:false
del C:\Users\Public\cmdlab_task.txt
```

Expected result. Task events 106 (registered), 200/201 (started/completed) and System event 7045 for `CmdLabSvc`. **Explanation.** The same benign actions an attacker would take leave these traces. **Verification.** Match event times to the order of your commands. **Troubleshooting.** If no task events appear, history was not enabled before the actions; repeat. **Cleanup.** Included in the script (history disabled again). **Challenge.** Write the three-line detection note: what event, what field, what is suspicious. **Solution.** 7045 `ImagePath` pointing to `cmd.exe` or a user-writable folder; 106 for a task registered by an unexpected user; 200 with an unusual `ActionName`.

Part XVI - Windows Defender

Reading Microsoft Defender Antivirus state from the command line, and knowing which commands only read and which change protection.

Chapter 37. Defender Administration and Investigation

37.1 Components and commands

Microsoft Defender Antivirus runs as service `WinDefend` (with `WdNisSvc` for network inspection) and is managed by policy, the Windows Security app, PowerShell (`Defender` module), and `MpCmdRun.exe`. **Tamper Protection**, when on, blocks changes to key settings through scripts, registry edits and command lines; it is a protection, and attempts to bypass it are alerts, not obstacles to engineer around.

The command-line binary `MpCmdRun.exe` is installed in the Defender platform folder (the copy under `%ProgramFiles%\Windows Defender` forwards to the current platform version under `%ProgramData%\Microsoft\Windows Defender\Platform\<version>`); `MpCmdRun.exe -h` lists the options for your build.

Operation	Command	Changes state?
Service state	`sc query WinDefend`	No
Overall status	`powershell -NoProfile -Command "Get-MpComputerStatus"`	No
Signature age	same cmdlet: `AntivirusSignatureLastUpdated`, `AntivirusSignatureVersion`	No
Detection history	`powershell -NoProfile -Command "Get-MpThreatDetection"`	No
Current settings / exclusions	`powershell -NoProfile -Command "Get-MpPreference"` (admin shows exclusions)	No
Event log review	`wevtutil qe "Microsoft-Windows-Windows Defender/Operational" /c:20 /rd:true /f:text`	No
Update signatures	`MpCmdRun.exe -SignatureUpdate`	**Yes** (downloads)
Quick scan	`MpCmdRun.exe -Scan -ScanType 1`	**Yes** (starts activity, load)
Scan one file or folder	`MpCmdRun.exe -Scan -ScanType 3 -File "path"`	Yes (activity)
Change settings	`Set-MpPreference`, `Add-MpPreference -ExclusionPath`	**Yes, weakens or alters protection**
Remove definitions	`MpCmdRun.exe -RemoveDefinitions ...`	**Yes, damages protection; do not use outside vendor support**

Scan types: 1 quick, 2 full, 3 custom (with `-File`). Quick scans finish in minutes; full scans take hours.

37.2 What to check in a health review

1. `AMServiceEnabled`, `AntivirusEnabled`, `RealTimeProtectionEnabled`, `BehaviorMonitorEnabled`, `IoavProtectionEnabled`, `IsTamperProtected` are all `True` (if managed by a third-party AV, Defender may be passive; confirm that product is healthy).
2. Signatures updated within the last day or two.
3. **Exclusions** reviewed: broad exclusions (whole drives, `C:\Users`, `*.exe`, temp folders) are risk. Exclusions added recently, by a non-admin process, or unexplained, are investigation items (event 5007 records changes).
4. Recent detections reviewed and their actions confirmed (1116 detection, 1117 action).
5. Real-time protection was never disabled unexpectedly (event 5001).

SECURITY. Defender status is evidence. If protection was disabled or an exclusion was added right before suspicious activity, preserve events and the `Get-MpPreference` output **before** remediation. Do not repair first and ask questions later.

37.3 Safe testing

To test that detection works without real malware, use the industry-standard **EICAR test file** (a harmless text file defined by the European Institute for Computer Antivirus Research). Obtain it and its documented content from **eicar.org**, in a VM; it is expected to be blocked immediately, which proves real-time protection and event logging work. This book does not reproduce the test string because many security products will flag the PDF itself.

Lab 156. Defender status check (read-only)

Objective. Report protection state without changing it. **Difficulty.** [I]. **Prerequisites.** Section 37.1. **Environment.** Elevated prompt.

Commands.

```
sc query WinDefend
powershell -NoProfile -Command "Get-MpComputerStatus | Select-Object AMServiceEnabled,AntivirusEnabled,RealTimeP
powershell -NoProfile -Command "Get-MpPreference | Select-Object -ExpandProperty ExclusionPath"
```

Expected result. Service `RUNNING`; the listed properties `True` with a recent signature date; an empty or short exclusion list. **Explanation.** Three independent facts: the service, the engine state, and the policy. **Verification.** Compare with Windows Security > Virus & threat protection. **Troubleshooting.** If a third-party AV is present, Defender may report passive mode. **Cleanup.** None. **Challenge.** Flag stale signatures. **Solution.** `powershell -NoProfile -Command "if ((New-TimeSpan (Get-MpComputerStatus).AntivirusSignatureLastUpdated).Days -gt 3) {'STALE'} else {'OK'}"`.

Lab 157. Defender events and a safe detection test (VM)

Objective. Confirm detections create events, using the EICAR test file. **Difficulty.** [I]. **Prerequisites.** Lab 156. **Environment. VM + snapshot** only.

Commands. Download the EICAR test file from eicar.org into a scratch folder using the browser (it will be blocked or quarantined), then:

```
wevtutil qe "Microsoft-Windows-Windows Defender/Operational" "/q:*[System[(EventID=1116 or EventID=1117)]]" /c:4
powershell -NoProfile -Command "Get-MpThreatDetection | Select-Object ThreatID,InitialDetectionTime,ActionSucces
```

Expected result. Events 1116 (detected) and 1117 (action taken) naming the test threat. **Explanation.** The same logs show real detections. **Verification.** Times match your download. **Troubleshooting.** No events means Defender is passive or real-time protection is off. **Cleanup.** Remove any leftover file; do not add exclusions. **Challenge.** Write the "what to preserve" list for a real detection. **Solution.** Event 1116/1117 XML, `Get-MpThreatDetection` output, the file path, hash, user and timestamp, then remediate.

Part XVII - Windows Repair and Recovery

System file verification, component store servicing, disk checks, boot configuration, WinRE and BitLocker, with every destructive command marked.

Chapter 38. Repair Tools and Boot Architecture

38.1 Concepts

- **System file integrity.** Protected OS files are verified against known-good copies in the **component store** (`C:\Windows\WinSxS`). `sfc` compares files with the store; **DISM** repairs the store itself.
- **Servicing.** Windows updates are installed as components; the store keeps multiple versions. `DISM /Cleanup-Image` options inspect and trim it.
- **Boot sequence (UEFI/GPT, typical):** firmware loads the **EFI System Partition (ESP)** boot manager (`bootmgfw.efi`), which reads the **Boot Configuration Data (BCD)** store, which points to `winload.efi` on the Windows volume, which loads the kernel. **BIOS/MBR** systems use the MBR and an active partition instead. The BCD is edited with `bcdedit`; boot files are recreated with `bcdboot`.
- **WinRE (Windows Recovery Environment):** a minimal Windows image (`Winre.wim`) with a Command Prompt, used when Windows does not boot. Many repair commands are available there; drive letters in WinRE can **differ** from the running system (use `dir` to find the Windows volume).
- **BitLocker** encrypts a volume; boot or hardware changes can trigger a **recovery key** prompt.

DANGER / CAUTION. Recovery commands can make a bootable machine unbootable or destroy data. Take a backup or VM snapshot first. Read-only commands are marked **(read-only)** in the cards below; commands marked **DANGER** change boot, disk or encryption state.

SFC [External, system file checker]

Field	Detail
Purpose / Why	Verifies protected system files and replaces corrupted ones from the component store
Syntax	`sfc /scannow` ; `sfc /verifyonly` ; `sfc /scanfile=path` ; `sfc /verifyfile=path` ; offline: `sfc /scannow /offbootdir=C:\ /offwindir=C:\Windows`
Beginner	`sfc /verifyonly` (reports, changes nothing)
Intermediate	`sfc /scannow`
Advanced	`findstr /c:"[SR]" %windir%\Logs\CBS\CBS.log > sfc_details.txt`
Real-world	After crashes or odd system behavior, or malware cleanup
Output	`Windows Resource Protection did not find any integrity violations.` / `found corrupt files and successfully repaired them` / `found corrupt files but was unable to fix some of them`
Exit / Errors	`You must be an administrator running a console session in order to use the sfc utility.` ; `Windows Resource Protection could not start the repair service.`
Admin	Yes
Versions	Same; if the store itself is damaged, run DISM `/RestoreHealth` first, then `sfc` again

Field	Detail
Mistakes	Expecting `sfc` to fix application files or third-party drivers; stopping it midway; reading the on-screen result without checking CBS.log
Security	`sfc` repairs files, which can also remove forensic evidence of tampering with protected files; capture `CBS.log` and hashes before repairing during an investigation
Lab / Cleanup	Lab 158
Related	`DISM`, `chkdsk`
PowerShell	`Repair-WindowsImage` (DISM equivalent)

DISM [External, servicing]

Field	Detail
Purpose / Why	Inspects and repairs the Windows image and component store; manages features, packages and images
Syntax	`DISM /Online /Cleanup-Image /CheckHealth|/ScanHealth|/RestoreHealth [/Source:wim:path:index /LimitAccess]` ; `DISM /Online /Cleanup-Image /AnalyzeComponentStore` ; `DISM /Online /Get-Packages` ; `DISM /Online /Get-Features`
Key switches	`/CheckHealth` quick flag check; `/ScanHealth` deeper scan, no repair; `/RestoreHealth` **repairs** (may download from Windows Update); `/StartComponentCleanup` removes superseded components; `/ResetBase` makes cleanup **irreversible**
Beginner	`DISM /Online /Cleanup-Image /CheckHealth` **(read-only)**
Intermediate	`DISM /Online /Cleanup-Image /ScanHealth` **(read-only)**
Advanced	`DISM /Online /Cleanup-Image /RestoreHealth /Source:wim:D:\sources\install.wim:1 /LimitAccess` (offline source)
Real-world	Fixing a failed Windows Update or an `sfc` that cannot repair
Output	Progress bar, then `No component store corruption detected.` / `The component store is repairable.` / `The restore operation completed successfully.`
Exit / Errors	0 success; 3010 success, reboot needed; 87 bad parameter; 0x800f081f source files not found (supply `/Source`); 740 needs elevation
Admin	Yes
Versions	`/Online` repairs the running OS; image servicing for offline images needs a mounted image
Mistakes	Using `/ResetBase` before you are sure you will never uninstall updates; running `/RestoreHealth` with no connectivity and no source; interrupting servicing
Security	DISM logs at `%windir%\Logs\DISM\dism.log`; unexpected feature enable/disable (for example SMB1, Telnet) is a finding. `Get-Features` helps audit.
Lab / Cleanup	Lab 159
Related	`sfc`, `wusa`, `Get-WindowsOptionalFeature`
PowerShell	`Repair-WindowsImage -Online -ScanHealth`, `Get-WindowsOptionalFeature -Online`

CHKDSK [External, disk check]

Field	Detail
Purpose / Why	Checks file system integrity and optionally repairs it or locates bad sectors
Syntax	`chkdsk [drive:] [/f] [/r] [/scan] [/spotfix] [/x]`
Key switches	no switch: **read-only** report; `/scan` online scan (NTFS); `/spotfix` quick offline repair of logged issues; `/f` **fixes errors**; `/r` **locates bad sectors and recovers data (implies `/f`, can take hours)**; `/x` force dismount
Beginner	`chkdsk D:` (read-only)
Intermediate	`chkdsk C: /scan`
Advanced	`chkdsk C: /spotfix` (offline, at next reboot for the system drive)
Real-world	After unexpected power loss or when the volume is flagged dirty
Output	Stages: file structure, indexes, security descriptors, then a summary of KB totals and bad sectors. Messages `Windows has scanned the file system and found no problems.` or `errors found`.
Exit / Errors	0 no errors; 1 errors found and fixed; 2 cleanup performed or not fully checked; 3 could not check or fix
Admin	Yes
Versions	ReFS uses different tools; for SSDs prefer `/scan` over frequent `/r`
Mistakes	Running `/r` on a **failing** disk (stress can finish it: image or back it up first); interrupting `/f`; reading "Convert lost chains" prompts as harmless
Security	Repairs change metadata; collect evidence before running on a disk under investigation
Lab / Cleanup	Read-only variants only in labs
Related	`fsutil dirty query C:`, `defrag` (HDD only), `Optimize-Volume`
PowerShell	`Repair-Volume -Scan`, `Repair-Volume -OfflineScanAndFix`

BCDEDIT, BCDBOOT, BOOTREC and REAGENTC [External, boot and recovery]

Field	Detail
Purpose / Why	`bcdedit`: view/edit Boot Configuration Data. `bcdboot`: (re)create boot files from a Windows folder. `bootrec`: legacy boot repair in WinRE. `reagentc`: manage the Recovery Environment.
Syntax	`bcdedit /enum [{current}|all]` **(read-only)** ; `bcdedit /export file` (backup) ; `bcdedit /import file` **DANGER** ; `bcdboot C:\Windows /s S: /f UEFI` **DANGER** ; `bootrec /scanos` (read-only), `/rebuildbcd`, `/fixmbr`, `/fixboot` **DANGER** ; `reagentc /info` (read-only), `/enable`, `/disable`
Beginner	`bcdedit /enum` ; `reagentc /info`
Intermediate	`bcdedit /export "%USERPROFILE%\bcd_backup"`
Advanced	In WinRE after verifying the Windows volume letter and mounting the ESP: `bcdboot D:\Windows /s S: /f UEFI`

Field	Detail
Real-world	Machine reports "boot device not found" after disk cloning or partition change
Output	`bcdedit`: identifiers (`{bootmgr}`, `{current}`), `device`, `osdevice`, `path` (`\Windows\system32\winload.efi`), `bootmenupolicy`, `safeboot` if set. `reagentc /info`: `Windows RE status: Enabled`, location `\\?\GLOBALROOT\device\harddisk0\partition4\Recovery\WindowsRE`.
Exit / Errors	`The boot configuration data store could not be opened. Access is denied.` (not elevated); `Element not found`; `bootrec /fixboot` often returns `Access is denied.` on UEFI systems (use `bcdboot` instead)
Admin	Yes; boot repair is normally done from WinRE or install media
Versions	UEFI/GPT and BIOS/MBR differ; **do not run MBR commands on a UEFI/GPT system**
Mistakes	Setting `bcdedit /set` values without a backup; removing the wrong entry; running `bootrec /fixmbr` on GPT; forgetting that WinRE letters differ
Security	Boot settings such as `testsigning on`, `nointegritychecks`, `debug on`, `safeboot` are security-relevant; `bcdedit /enum` **showing unexpected values (test signing, integrity checks off, kernel debug) is a finding.** Secure Boot status: `msinfo32` or PowerShell `Confirm-SecureBootUEFI`.
Lab / Cleanup	Lab 160 is read-only plus export
Related	`diskpart` (list only in labs), `mountvol`, `manage-bde`
PowerShell	`Confirm-SecureBootUEFI`, `Get-Disk`, `Get-Partition`

MANAGE-BDE [External, BitLocker]

Field	Detail
Purpose / Why	Shows and manages BitLocker drive encryption
Syntax	`manage-bde -status [C:]` **(read-only)** ; `manage-bde -protectors -get C:` (shows protector types; **may display the recovery password**) ; `manage-bde -on C:` **DANGER** ; `manage-bde -off C:` **DANGER** ; `-lock`, `-unlock`, `-pause`, `-resume`
Beginner	`manage-bde -status`
Intermediate	`manage-bde -status C: | findstr /i "Conversion Percentage Protection"`
Advanced	`manage-bde -protectors -get C: -type recoverypassword` (treat output as a secret)
Real-world	Auditing which fixed drives are encrypted; confirming a key protector exists before hardware changes
Output	`Conversion Status` (Fully Encrypted / Used Space Only Encrypted / Fully Decrypted), `Percentage Encrypted`, `Encryption Method` (for example `XTS-AES 128`), `Protection Status` (On/Off), key protectors
Exit / Errors	`ERROR: An attempt to access ... failed. Access is denied`; BitLocker not available on Home (Device Encryption has its own UI)
Admin	Yes
Versions	Requires Pro/Enterprise/Education for full BitLocker; Home may have device encryption on supported hardware

Field	Detail
Mistakes	**Pasting a recovery password into a ticket, chat or log**; suspending/disabling protection and forgetting to resume; turning BitLocker on without escrowing the recovery key
Security	Back up recovery keys to a secured location (Entra ID/AD/account or offline vault). `Protection Status: Off` on a laptop is a finding.
Lab / Cleanup	Lab 161 (status only)
Related	`cipher`, `reagentc`, `msinfo32`
PowerShell	`Get-BitLockerVolume`, `Backup-BitLockerKeyProtector`

Lab 158. sfc verify-only and reading CBS.log

Objective. Verify without repairing, then read the evidence. **Difficulty.** [I]. **Prerequisites.** `sfc` card. **Environment. VM**, elevated.

Commands.

```
sfc /verifyonly
findstr /c:"[SR]" %windir%\Logs\CBS\CBS.log > sfc_details.txt
find /c /v "" sfc_details.txt
findstr /i "Cannot repair" sfc_details.txt
```

Expected result. `did not find any integrity violations` on a healthy system; a detail file of `[SR]` lines; no `Cannot repair` lines. **Explanation.** `/verifyonly` is safe; the log explains any failure. **Verification.** The count is nonzero (verification entries are logged). **Troubleshooting.** The run takes several minutes; do not interrupt. **Cleanup.** `del sfc_details.txt`. **Challenge.** Decide the next step if violations exist. **Solution.** Run `DISM /Online /Cleanup-Image /RestoreHealth`, reboot, then `sfc /scannow`, then recheck the log.

Lab 159. DISM health checks (read-only)

Objective. Check component store health and package inventory. **Difficulty.** [I]. **Prerequisites.** `DISM` card. **Environment. VM**, elevated.

Commands.

```
DISM /Online /Cleanup-Image /CheckHealth
DISM /Online /Cleanup-Image /AnalyzeComponentStore
DISM /Online /Get-Packages /Format:Table | more
```

Expected result. `No component store corruption detected.`; analysis reporting store size and `Component Store Cleanup Recommended : Yes/No`; a table of installed packages. **Explanation.** Inspection first, repair only if flagged. **Verification.** Exit code 0 (`echo %ERRORLEVEL%` on its own line). **Troubleshooting.** Error 740 means not elevated. **Cleanup.** None. **Challenge.** Find recently installed updates. **Solution.** `DISM /Online /Get-Packages /Format:Table` and read the `Install Time` column.

Lab 160. Read boot configuration and back it up

Objective. Record the BCD and recovery environment state. **Difficulty.** [I]. **Prerequisites.** `bcdedit` card. **Environment. VM**, elevated.

Commands.

```
bcdedit /enum
bcdedit /enum {current}
bcdedit /export "%USERPROFILE%\bcd_backup"
reagentc /info
bcdedit /enum | findstr /i "testsigning nointegritychecks debug safeboot"
```

Expected result. Boot manager and loader entries; a backup file created; WinRE `Enabled`; the final `findstr` ideally prints nothing. **Explanation.** The backup is your undo for any later boot change; security-relevant flags should be absent. **Verification.** `dir "%USERPROFILE%\bcd_backup"`. **Troubleshooting.** `Access is denied` means not elevated. **Cleanup.** Keep the backup in a safe place. **Challenge.** Explain what you would do if `testsigning Yes` appeared. **Solution.** Identify who changed it and why (change record, event logs, drivers); if unauthorized, treat as an incident, then restore the backed-up BCD.

Lab 161. Encryption and recovery posture (read-only)

Objective. Report BitLocker and WinRE status without exposing secrets. **Difficulty.** [I]. **Prerequisites.** `manage-bde` card. **Environment. VM** or personal PC, elevated.

Commands.

```
manage-bde -status C:
manage-bde -status C: | findstr /i "Conversion Protection Method"
reagentc /info | findstr /i "status location"
```

Expected result. Conversion and protection state, encryption method, WinRE status and location. **Explanation.** These two facts decide whether a lost laptop exposes data and whether recovery tools are available. **Verification.** Do **not** run `-protectors -get` on a screen others can see. **Troubleshooting.** On Home the command may report BitLocker unavailable. **Cleanup.** None. **Challenge.** Write a one-line audit rule. **Solution.** `Protection Status` must be `On` for fixed drives on mobile devices; WinRE must be `Enabled`.

Part XVIII - Windows Performance

Measuring CPU, memory, disk, network, processes, services, startup and power from CMD, and building a repeatable triage.

Chapter 39. Performance Measurement and Triage

39.1 The four resources and what "slow" means

Resource	Symptom	Counter or tool	Rough guideline (context matters)
CPU	Fan noise, sluggish everything	`\Processor(_Total)\% Processor Time`	Sustained above ~85% needs explanation
Memory	Swapping, apps closing	`\Memory\Available MBytes`, `\Memory\Pages/sec`	Available near zero with heavy paging means pressure
Disk	Long freezes, high latency	`\PhysicalDisk(_Total)\Avg. Disk Queue Length`, `% Disk Time`, `Avg. Disk sec/Read`	Queue sustained above ~2 per spindle, latency above ~20-25 ms on HDD
Network	Slow transfers	`\Network Interface(*)\Bytes Total/sec`	Compare with adapter speed

Method: **measure, identify the bottleneck, find the process, decide**. Never "tune" without a before/after measurement. A busy resource is not automatically a problem: 100% CPU during a build is normal.

DANGER / CAUTION. Percent signs in counter names. Counter names such as `\Processor(_Total)\% Processor Time` contain a `%`. In a **batch file** you must write `%%` (as Lab 165 does) or CMD will treat the text up to the next `%` as a variable name and silently delete it. At the **prompt**, avoid putting such a counter on the same line as a `%VARIABLE%` for the same reason (use a counter without a percent sign, or a literal path).

Finding heavy processes:

```
tasklist /v /fo csv /nh
powershell -NoProfile -Command "Get-Process | Sort-Object WorkingSet64 -Descending | Select-Object -First 10 Name
powershell -NoProfile -Command "Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 Name,Id,CPU"
```

Startup review:

```
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
dir "%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup"
schtasks /query /fo csv | findstr /i "ONLOGON ONSTART"
powershell -NoProfile -Command "Get-CimInstance Win32_StartupCommand | Select-Object Name,Command,Location"
```

(`wmic startup` is the old way and is deprecated.)

TYPEPERF and LOGMAN [External, performance counters]

Field	Detail
Purpose / Why	`typeperf`: print live counter values. `logman`: create, start, stop and delete background data collectors.
Syntax	`typeperf "\Processor(_Total)\% Processor Time" [-si seconds] [-sc samples] [-o file -f CSV]` ; `typeperf -q Memory` (list counters) ; `logman create counter NAME -c "counter" -si 5 -o path` ; `logman start NAME` ; `logman stop NAME` ; `logman query [NAME]` ; `logman delete NAME`
Key switches	`-si` sample interval seconds, `-sc` sample count, `-o` output file, `-f CSV` format; `logman -max` size limit, `-rf hh:mm:ss` run for
Beginner	`typeperf "\Memory\Available MBytes" -sc 3`
Intermediate	`typeperf "\Processor(_Total)\% Processor Time" "\Memory\Available MBytes" -si 2 -sc 10 -o perf.csv -f CSV`
Advanced	`logman create counter CmdLabPerf -c "\Memory\Available MBytes" -si 5 -rf 00:01:00 -o "%TEMP%\cmdlabperf"` then `logman start CmdLabPerf`
Real-world	Capturing a baseline during a quiet period and during a complaint, then comparing
Output	CSV: timestamp column, then counter columns; the last line reports `Exiting, please wait... The command completed successfully.`
Exit / Errors	`Error: No valid counters.` (misspelled path or counter missing; list with `-q`); counter names depend on the system language
Admin	`logman` create/start usually yes; `typeperf` for local counters no
Versions	Same; non-English systems need localized counter names
Mistakes	Forgetting quotes around counter paths; leaving a collector running; collecting very short intervals for days
Security	Collector sets are a persistent service; an unexpected one is worth checking (`logman query`)
Lab / Cleanup	Labs 162, 164; `logman delete` at the end
Related	`tasklist`, `perfmon`, `relog`
PowerShell	`Get-Counter`, `Get-CimInstance Win32_PerfFormattedData_*`

POWERCFG [External, power management]

Field	Detail
Purpose / Why	Inspects and manages power plans and diagnoses sleep, battery and energy problems
Syntax	`powercfg /list` ; `powercfg /getactivescheme` ; `powercfg /a` (available sleep states) ; `powercfg /requests` (what blocks sleep; admin) ; `powercfg /energy [/output file] [/duration s]` (admin) ; `powercfg /batteryreport` ; `powercfg /sleepstudy` ; `powercfg /setactive GUID` **(changes system behavior)**
Beginner	`powercfg /list`
Intermediate	`powercfg /a`
Advanced	`powercfg /energy /output "%TEMP%\energy.html" /duration 20`

Field	Detail
Real-world	Finding why a laptop will not sleep or drains overnight
Output	`/list`: GUID and name of each plan with `*` on the active one. `/energy` writes an HTML report of errors, warnings and informational findings.
Exit / Errors	`You do not have permission to run this command` (needs elevation)
Admin	Many options yes; `/list` and `/a` no
Versions	`/sleepstudy` and modern-standby reports exist on modern-standby hardware only
Mistakes	Using `/energy` while the machine is busy (results are noisy); disabling hibernation (`/h off`) without understanding Fast Startup effects
Security	Power settings that disable the lock screen timeout or enable wake from network may violate policy
Lab / Cleanup	Lab 163
Related	`systeminfo`, `typeperf`
PowerShell	No direct cmdlet; `powercfg.exe` remains the tool

Lab 162. Sample counters to CSV

Objective. Collect CPU and memory samples and read the result. **Difficulty.** [I]. **Prerequisites.** `typeperf` card. **Environment.** Scratch folder.

Commands.

```
typeperf "\Processor(_Total)\% Processor Time" "\Memory\Available MBytes" -si 2 -sc 10 -o perf.csv -f CSV
type perf.csv
```

Expected result. A CSV with 10 rows of timestamps and two counter values. **Explanation.** Two metrics sampled together let you correlate CPU with memory. **Verification.** `find /c /v "" perf.csv` shows 11 lines (header plus 10). **Troubleshooting.** `No valid counters` on a localized system: use `typeperf -q Processor` to find your language's name. **Cleanup.** `del perf.csv`. **Challenge.** Generate load and watch the CPU counter rise. **Solution.** In a second prompt run `for /l %i in (1,1,200000) do @rem` (a busy loop) while sampling.

Lab 163. Power diagnostics

Objective. Read power configuration and sleep blockers. **Difficulty.** [I]. **Prerequisites.** `powercfg` card. **Environment.** Elevated prompt.

Commands.

```
powercfg /getactivescheme
powercfg /a
powercfg /requests
powercfg /energy /output energy.html /duration 20
```

Expected result. Active plan GUID and name; supported sleep states; requests by DISPLAY/SYSTEM/AWAYMODE (usually `None`); an HTML report file. **Explanation.** `/requests` names the

process preventing sleep. **Verification.** Open `energy.html` in a browser and read the Errors section. **Troubleshooting.** Close busy applications before `/energy`. **Cleanup.** `del energy.html`. **Challenge.** Start a video and re-run `/requests`. **Solution.** The player appears under DISPLAY and SYSTEM requests.

Lab 164. Background collector lifecycle

Objective. Create, run, inspect and delete a counter collector. **Difficulty.** [A]. **Prerequisites.** Lab 162. **Environment. VM**, elevated.

Commands.

```
logman create counter CmdLabPerf -c "\Memory\Available MBytes" -si 2 -rf 00:00:20 -o "%TEMP%\cmdlabperf"
logman start CmdLabPerf
logman query CmdLabPerf
timeout /t 25 /nobreak >nul
logman query CmdLabPerf | findstr /i "Status"
dir "%TEMP%\cmdlabperf*"
logman delete CmdLabPerf
```

Expected result. Collector `Running` then `Stopped` after 20 seconds; a `.blg` file in TEMP; the delete succeeds. **Explanation.** Data collector sets run in the background and persist until deleted. **Verification.** `logman query` no longer lists `CmdLabPerf`. **Troubleshooting.** `Access is denied` means not elevated. **Cleanup.** `del "%TEMP%\cmdlabperf*"`. **Challenge.** Convert the `.blg` to CSV. **Solution.** `relog "%TEMP%\cmdlabperf.blg" -f CSV -o out.csv` (the exact file name depends on the collector's file-versioning settings; check with `dir "%TEMP%\cmdlabperf*"`).

Lab 165. Performance triage report script

Objective. Collect a one-shot performance snapshot into a file. **Difficulty.** [A]. **Prerequisites.** Labs 162-164. **Environment.** Scratch folder.

Commands. Save as `perf_triage.cmd`:

```
@echo off
setlocal EnableExtensions
set "OUT=%~dp0perf_triage.txt"
> "%OUT%" echo Performance triage %COMPUTERNAME% %DATE% %TIME%
>> "%OUT%" echo ---- uptime and memory (systeminfo)
systeminfo | findstr /b /c:"System Boot Time" /c:"Total Physical Memory" /c:"Available Physical Memory" >> "%OUT%
>> "%OUT%" echo ---- counters (5 samples)
typeperf "\Processor(_Total)\%% Processor Time" "\Memory\Available MBytes" "\PhysicalDisk(_Total)\Avg. Disk Queue
>> "%OUT%" echo ---- top memory processes
powershell -NoProfile -Command "Get-Process | Sort-Object WorkingSet64 -Descending | Select-Object -First 8 Name,
>> "%OUT%" echo ---- startup commands
powershell -NoProfile -Command "Get-CimInstance Win32_StartupCommand | Select-Object Name,Location | Format-Table
type "%OUT%"
exit /b 0
```

Expected result. A readable report with five sections. **Explanation.** One command that produces comparable snapshots before and after a change is the foundation of performance work. **Verification.** Run it on a quiet machine, then under load, and compare the counters. **Troubleshooting.** Localized counter names break the `typeperf` line; adjust from `typeperf -q`. **Cleanup.** `del perf_triage.cmd perf_triage.txt`. **Challenge.** Add a pass/fail threshold for available memory. **Solution.** Capture the counter into a variable with `for /f`,

compare with `if`, and write `WARN low memory`.

Back Matter (Volume 2)

Glossary additions, command index, references, items to verify, and the plan for Volumes 3 and 4.

Chapter 40. Glossary Additions

Term	Definition
ACE / ACL	Access control entry / list: an ordered set of allow or deny entries on an object
ADS	Alternate data stream: a named extra data stream attached to an NTFS file
BCD	Boot Configuration Data: the store that tells the boot manager what to boot
Channel	A named event log stream (Security, System, Microsoft-Windows-...)
Component store	`WinSxS`: the repository of Windows components used by servicing and `sfc`
DACL / SACL	Discretionary ACL (who may access) / system ACL (what is audited and integrity labels)
EFS	Encrypting File System: per-file NTFS encryption tied to a user certificate
ESP	EFI System Partition holding UEFI boot files
Filtered token	The reduced token an administrator receives under UAC for normal programs
Hive	A registry file loaded at a key such as `HKLM\SOFTWARE` or `HKU\<SID>`
Hard link	An additional directory entry for the same file data on one NTFS volume
Integrity level	Token and object label (Low, Medium, High, System) enforced by mandatory integrity control
Junction	A directory reparse point redirecting to another local folder
MFT	NTFS Master File Table: one record per file or folder
MTU	Largest packet size on a link
NAT	Rewriting of private addresses to a public one at a router
Owner	The SID that may always change an object's DACL
Principal (Task Scheduler)	The identity and run level under which a task executes
Privilege	A system-wide right held in a token (for example `SeBackupPrivilege`); distinct from a permission
Provider (event)	The component that writes events into channels
Reparse point	A file or folder with a tag telling the file system driver to involve another component
RID	Relative identifier: the last number of a SID (500 = built-in Administrator)
SCM	Service Control Manager (`services.exe`)
SID	Security identifier: the unique internal name of a user, group or service identity
Session 0	The non-interactive session where services run
Tamper Protection	Defender feature blocking unauthorized changes to protection settings
TTL	Time to live: hop limit in IP packets and cache lifetime in DNS
UNC	Universal Naming Convention path (`\\server\share`)
WinRE	Windows Recovery Environment

Chapter 41. Command Index (Volume 2)

Command	Where
`arp`, `route`, `getmac`	Chapter 33
`bcdedit`, `bcdboot`, `bootrec`, `reagentc`	Chapter 38
`chkdsk`	Chapter 38
`cipher`, `compact`	Chapter 24
`DISM`	Chapter 38
`eventcreate`	Chapter 36
`fsutil`	Chapter 24
`icacls`, `takeown`	Chapter 24
`ipconfig`	Chapter 33
`logman`, `typeperf`	Chapter 39
`manage-bde`	Chapter 38
`net user`, `net localgroup`	Chapter 29
`net start/stop/pause/continue`	Chapter 27
`netsh`	Chapter 33
`netstat`	Chapter 33
`nslookup`	Chapter 33
`ping`, `tracert`, `pathping`	Chapter 33
`powercfg`	Chapter 39
`reg`	Chapter 30
`robocopy`	Chapter 24
`runas`	Chapter 29
`sc`	Chapter 27
`schtasks`	Chapter 31
`setx`	Chapter 30
`sfc`	Chapter 38
`systeminfo`	Chapter 25
`tasklist`, `taskkill`	Chapter 25
`wevtutil`	Chapter 36
`whoami`	Chapter 29
`MpCmdRun.exe` (Defender)	Chapter 37 (table, not full card)

Still to be carded in later volumes: `certutil`, `bitsadmin`, `regsvr32`, `rundll32`, `mshta`, `wmic`, `forfiles`, `msiexec`, `installutil`, `cscript`, `wscript`, `driverquery`, `query`, `quser`, `qwinsta`, `openfiles`, `shutdown`,

`diskpart`, `mountvol`, `format`, `label`, `convert`, `defrag`, `expand`, `makecab`, `replace`, `telnet`, `ftp`, `mode`, `break`, `verify`, `subst`.

Chapter 42. References and Verification Notes

Primary sources (verify each command against the page for your exact Windows build):

- Microsoft Learn, *Windows commands* reference, including pages for `icacls`, `takeown`, `fsutil`, `cipher`, `compact`, `robocopy`, `tasklist`, `taskkill`, `sc`, `net user`, `net localgroup`, `whoami`, `runas`, `reg`, `schtasks`, `ipconfig`, `ping`, `tracert`, `pathping`, `nslookup`, `netstat`, `arp`, `route`, `netsh`, `getmac`, `wevtutil`, `eventcreate`, `sfc`, `DISM`, `chkdsk`, `bcdedit`, `bcdboot`, `bootrec`, `reagentc`, `manage-bde`, `typeperf`, `logman`, `powercfg`.
- Microsoft Learn, *Windows security*: access tokens, security descriptors and ACLs, User Account Control, mandatory integrity control, Windows Defender Firewall, Microsoft Defender Antivirus, BitLocker.
- Microsoft Learn, *Windows Security Auditing* event reference (4624, 4625, 4688, 4697, 4698, 4720, 4732, 1102 and others) and *Advanced security audit policy* settings.
- Microsoft Sysinternals documentation: Process Explorer, Process Monitor, Sysmon, Autoruns, AccessChk, Streams, TCPView.
- MITRE ATT&CK: T1053.005 (Scheduled Task), T1543.003 (Windows Service), T1547.001 (Registry Run Keys / Startup Folder), T1070.001 (Clear Windows Event Logs), T1564.004 (NTFS File Attributes / ADS), T1562.004 (Disable or Modify System Firewall), T1562.001 (Impair Defenses). Used here only as vocabulary for detection and audit.
- EICAR (eicar.org): standard anti-malware test file documentation.

Items to verify on your own machine before relying on them. These were written from documented behavior and widely reported findings; I have not executed any command or lab on Windows.

- Exact output text and column layouts (they vary by build and language).
- Whether `runas` against a nonexistent account produces event 4625 (Lab 152).
- Whether `schtasks /create /sc onlogon` needs elevation on your build (Lab 100).
- `netsh advfirewall firewall show rule ... status=enabled` parameter support (Lab 111).
- The log file naming produced by `logman` (Lab 164).
- Defender cmdlet property names and the platform path of `MpCmdRun.exe` (Chapter 37).
- Availability and default state of `wmic` on Windows 11 24H2 and later.
- The `forfiles` stale-lock approach (Section 22.5) and the `fltmc` admin test (Section 22.9) on hardened systems.

Safety recap. Every lab that changes state says so, uses a disposable name (`CmdLab...`), and includes cleanup. Run labs in a VM with a snapshot. The DANGER-marked commands (`bootrec /fixboot`, `bcdedit /import`, `manage-bde -on/-off`, `reg restore`, `robocopy /mir`, `chkdsk /r`, `DISM /ResetBase`, `wevtutil cl`) are described so you can recognize them and rehearse safely; none appear in a lab as an unprotected step.

Chapter 43. What Comes Next

Volume 3 (planned): defensive security research (Part XIX), command injection and batch security (XX), PATH and command-resolution security (XXI), incident response (XXII) and forensics (XXIII), all on benign simulations; the security labs (116-135) and incident-response labs (136-145); the deferred cards for `certutil`, `bitsadmin`, `regsvr32`, `rundll32`, `mshta`, `wmic`, `forfiles`, `msiexec`, `installutil`, `cscript` and `wscript`; and the remaining batch labs (71-80).

Volume 4 (planned): networking and administration labs not yet written (113-115 and any gaps), expert labs 146-150, the 17 projects, the capstone audit toolkit, the troubleshooting encyclopedia, expert reference tables, CMD vs PowerShell, the combined glossary and master command index.